

REFERENCE SNAPSHOT · COMPLETE DOCUMENTATION

Predicting D52 dopaminergic differentiation efficiency from D11

The full written record of the D11 → D52 prototype: overview, findings, modeling methods and EDA provenance, in one document.

REPOSITORY	pluricon-prototype
COMMIT	fe45c45db031c9d8c3224ed4bfbf245f1378ced6
GENERATED	2026-09-11
CONTENTS	README.md · FINDINGS.md · modeling/README.md · metadata_eda/README.md
DATASET	Jerber et al. 2021, <i>Nat Genet</i> 53:304–312 — 138 iPSC lines

Captured at commit `fe45c45`, before the repository is extended to the D30 → D52 model. Every section of all four source documents is reproduced verbatim; nothing is summarised or omitted.

Contents

Part I · Project overview

README.md

Data
Features (D11 only)
Cross-validation
Models and hyperparameters
Results

Which features matter
Technical confounders — read before interpreting any PC
Repository
Setup

Part II · Findings and literature assessment

FINDINGS.md

1. Model performance
2. Which features are informative
3. Technical confounders
4. Additional findings worth recording

5. Literature assessment: recapitulated vs. new
 The important caveat on finding #2
6. What would strengthen this most
7. What this does not show
Sources

Part III · Modeling methods and full results

modeling/README.md

Layout
First-pass prototype scope (decided while implementing)
Full results (258 folds: plain 50, donor_grouped 50, LOCO 138, LODO 20)
Which features matter (feature importance)
 What the columns mean
 What the "one-SE rule" is
 Why the tables stop at PC5 (regression) / PC2 (classification)
Fitting population: the qualifying-only PCA variant
 Do NOT compare the two importance tables row by row
Corrections from the audit
Second correctness review (variant plumbing + test pollution)
D30 readiness — NOT ready (read before starting D30 -> D52)
What PC1 is: a D11 proliferation axis
Pool-correction dropped
Setup
 Hyperparameters: 55 configurations per model
CV — both variants run, neither "primary"
 How the 138 lines are split, by scheme
 Flat vs nested: which is used for what

How nested CV actually runs (one repeat)
How flat CV runs -- and one thing it does NOT do
Best flat-CV configurations (donor_grouped)
Reconciling the two: they produce different things
What donor_grouped actually does
Leakage safety
Metrics
 Classification
 Regression
Reproducibility
Scope/cost warning
Results distillation
Architecture (Open-Closed + reusable for D30→D52)
Files
Verify
Latest results
 Headline (pre-registered model per task)
 Configurations selected
 Hyperparameter selection is unstable -- read single-config tables with that in mind
 Files backing this section

Part IV · EDA outputs and provenance

metadata_eda/README.md

Layout
Source metadata fields (in the .h5 files themselves)
From 001_eda.py
From 002_metadata_eda.py
From 003_qualifying_cell_lines.py
From 004_d11_celltype_proportion_se.py
From 005_d11_pca_features.py

From 006_d11_pca_variance_vs_se.py
From 007_d11_cell_counts_per_line.py
From 008_technical_covariate_associations.py
From 009_d52_outcome_label.py
From 010_d52_label_technical_covariates.py
Regenerating

Project overview

source: [README.md](#)

Root README: the question, the data, the pipeline in brief, and headline results.

pluricon-prototype

Predicting **day-52 dopaminergic neuron differentiation efficiency** from **day-11** single-cell features, across 138 iPSC lines.

Practical question: 11 days into a midbrain differentiation, can you tell whether the run will succeed — early enough to abort a bad one?

Short answer: yes. ROC-AUC 0.947, and 100% precision at 65% recall for detecting failures, under never-seen-donor cross-validation.

Full write-ups: [FINDINGS.md](#) (results + literature context) · [modeling/README.md](#) (methods) · [modeling/docs/PCA_interpretation.md](#) (what the PCs mean).

Data

[Jerber et al. 2021](#), *Nat Genet* 53:304–312 — scRNA-seq of iPSC lines differentiated toward midbrain dopaminergic fate, multiplexed into pools, sampled at D11, D30 and D52.

Lines used	138 (≥10 cells at D11 and D30 and D52), from 159 (line, pool) combos
Predictors	D11 only — 253,381 cells
Outcome	diff_efficiency = fraction of a line's D52 cells that are DA or Sert neurons
Binary label	success = efficiency ≥ 0.2 (the authors' threshold) → 96 success / 42 failure
Outcome shape	bimodal: failures 0–0.185, successes 0.221–0.92, real gap at the threshold

Features (D11 only)

1. **Cell-type proportions** — phat_FPP, phat_NB, phat_P_FPP. These sum to 1, so phat_P_FPP is the implicit reference and only two enter any fit.
 2. **PC1–PC10** — 2,000 HVGs → standardised → PCA. Per-cell coordinates averaged **pool-then-line** (mean per (line, pool), then unweighted mean of pool means).
- Both are computed **per fold**: HVG selection and the PCA are fit on training-line cells only, then *all* cells are projected — so no held-out line influences the basis it is scored in. pool11 is excluded from every fit (≈1,900 UMI/cell vs 10,000–18,000 elsewhere) but still projected, so no line is lost.

Cross-validation

Primary scheme	donor-grouped 5-fold, 10 repeats, stratified by the binary label (136/138 lines share a donor with another line)
Also run	plain 5-fold, LOCO (138 folds), LODO (20 folds)
Tuning	nested — inner 3-fold on training lines only; inner metric PR-AUC (classification) / MAE (regression)
Metrics	computed per repeat , then averaged — never pooled across repeats
Total	258 folds, PCA refit for each

Model per task was **pre-registered before looking at results**: donor-grouped + nested + ridge / logistic_l2.

Models and hyperparameters

Tuned jointly over `k` (number of PCs) and regularisation strength — 55 configurations per model.

model	task	grid
ridge <i>(headline)</i>	regression	<code>alpha</code> ∈ {0.01, 0.1, 1, 10, 100}
lasso	regression	<code>alpha</code> ∈ {0.001, 0.01, 0.1, 1, 10}
logistic_l2 <i>(headline)</i>	classification	<code>C</code> ∈ {0.01, 0.1, 1, 10, 100}
logistic_l1	classification	<code>C</code> ∈ {0.01, 0.1, 1, 10, 100}

`k` ∈ 0–10 for all. Both logistic models use `class_weight="balanced"` and a fixed seed. Features standardised on training folds only.

Results

Nested CV, mean ± SD across 10 repeats. `plain` ignores donors; `donor-grouped` never lets a donor appear in both train and test.

Classification (what every metric means, and why these: `modeling/README.md` → "Metrics")

model	scheme	ROC-AUC	PR-AUC	balanced acc	sensitivity	specificity	F1	Brier
logistic_l2 <i>(headline)</i>	plain	0.940 ± 0.012	0.964 ± 0.011	0.914 ± 0.013 ★	0.931 ± 0.016	0.898 ± 0.023 ★	0.943 ± 0.009	0.102 ± 0.006
logistic_l2 <i>(headline)</i>	donor-grouped	0.947 ± 0.007	0.968 ± 0.004	0.913 ± 0.013	0.940 ± 0.007	0.886 ± 0.022	0.945 ± 0.007 ★	0.097 ± 0.008
logistic_l1 <i>(secondary)</i>	plain	0.948 ± 0.011	0.968 ± 0.012	0.904 ± 0.019	0.933 ± 0.005	0.874 ± 0.037	0.939 ± 0.009	0.077 ± 0.007
logistic_l1 <i>(secondary)</i>	donor-grouped	0.951 ± 0.008 ★	0.970 ± 0.009 ★	0.912 ± 0.013	0.942 ± 0.011 ★	0.883 ± 0.021	0.945 ± 0.008 ★	0.075 ± 0.004 ★
trivial baseline	—	0.500	0.696	0.500	1.000	0.000	0.821	0.212

Regression

model	scheme	R²	MAE	RMSE	out-of-range
ridge <i>(headline)</i>	plain	0.676 ± 0.013	0.134 ± 0.003 ★	0.170 ± 0.003	0.050 ± 0.007
ridge <i>(headline)</i>	donor-grouped	0.653 ± 0.021	0.138 ± 0.004	0.175 ± 0.005	0.043 ± 0.005 ★
lasso <i>(secondary)</i>	plain	0.680 ± 0.016 ★	0.134 ± 0.003 ★	0.169 ± 0.004 ★	0.049 ± 0.008
lasso <i>(secondary)</i>	donor-grouped	0.668 ± 0.015	0.135 ± 0.003	0.172 ± 0.004	0.046 ± 0.004
predict the mean	—	0.000	0.266	0.298	0.000

★ = best in that column among the four fitted rows (baselines excluded). **Bold** marks the pre-registered headline row, which is a separate thing from being best. Higher is better everywhere except **Brier**, **MAE**, **RMSE** and **out-of-range**, where lower is better — so the star follows the minimum in those columns.

Baseline row = what a trivial model scores: always predicting "success" for classification (or random ranking, for the two AUCs), and predicting the training mean for regression. **Sensitivity 1.000 and F1 0.821 from that baseline are why neither is quoted alone** — with 70% successes, guessing "success" every time already looks respectable on both.

Best model: L2-logistic on the binary task. The L1 variants edge it but well within one SD, and were not pre-registered.

Donor grouping costs regression a little and classification nothing — ridge −0.023 R², lasso −0.011, while both classification models are flat or slightly better. LOCO/LODO agree (0.682/0.665 R², 0.942/0.963 AUC), so donor leakage is real but small, about 0.012 R².

⚠ **The regression R² mostly reflects separating the two outcome clumps, not fine-grained accuracy.** Within-success R² is only ~0.17–0.22 and within-failure R² is strongly negative. Use this for success/failure calls, not as an efficiency estimate.

The PCA fitting population barely matters. Refitting the whole pipeline on cells from the 138 study lines only shifts donor-grouped results by +0.014 R² (ridge), +0.004 (lasso), ±0.001 AUC — all inside one SD. It does roughly halve regression's across-repeat SD (0.021 → 0.012), so it is somewhat more stable, but no conclusion changes.

Which features matter

phat_NB, the D11 neuroblast fraction, is the **standout**. More neuroblasts at D11 → *worse* D52 yield.

evidence	value
Univariate Spearman ρ	−0.73 — strongest of anything measured
LOCO Δ (classification)	+0.051 AUC, largest single feature
SHAP (classification)	1.19 vs PC2's 0.51
Pool η^2	0.04 — the only technically clean feature

Dropping all proportions costs **+0.064 AUC** — the largest effect in either table — but costs regression nothing. On the restricted PCA basis the modal model selects **zero PCs** and still reaches AUC 0.950.

PC1 is a proliferation axis (ρ = +0.563; enriched for the Tirosh/Seurat G2/M set at p = 2.3×10^{-26} ; tracks proliferating-FPP fraction at ρ = +0.798). More proliferative D11 cultures differentiate better.

Technical confounders — read before interpreting any PC

Pool identity explains much of the variance in most PC features:

feature	pool η^2
phat_NB	0.04
PC1 / PC3 / PC4	0.45–0.50
PC2	0.76 — <i>and the most predictive PC</i>
PC6 / PC8 / PC9	0.91–0.96 — near-pure batch

So **PC2 is model-predictive but technically entangled**; it may not transfer to new pools. Separately, PC2 and PC3 are near-tied in variance and **rotate between PCA fits** — PC_n is not a stable label. PC1 is the exception (replicates at r = 0.9995).

Repository

FINDINGS.md	results + what is new vs. prior work
metadata_eda/	EDA outputs: cohort/ pca/ proportions/ qc/ technical/ plots/
modeling/	the pipeline (see modeling/README.md)
0NN_*.py	numbered EDA scripts, run in order

Setup

```
uv sync
uv run pytest modeling/ -m "not slow"      # 51 tests
uv run python -m modeling.run_classification
```

Python 3.11, [uv](#). Expression data (day11.h5 , day30.h5 , day52.h5) is not in the repo; set the paths in modeling/features.py .

Findings and literature assessment

source: FINDINGS.md

What was found, and which findings recapitulate Jerber et al. (2021) versus add something new.

Findings: predicting D52 dopaminergic differentiation efficiency from D11

Summary of results from this prototype, with an assessment of which findings **recapitulate** Jerber et al. (2021) or established stem-cell biology, and which appear to be **new**.

Data: Jerber et al. 2021 (*Nat Genet* 53:304–312), 138 iPSC lines with ≥ 10 cells at D11, D30 and D52. Outcome: `diff_efficiency` = fraction of a line's D52 cells that are DA or Sert neurons; "success" = ≥ 0.2 , the authors' own threshold. 96 successes / 42 failures.

Method detail lives in `modeling/README.md` ; PC biology in `modeling/docs/PCA_interpretation.md` .

1. Model performance

Nested CV, mean $\pm$ SD across 10 repeats. Model per task was **pre-registered before looking at results** to avoid cherry-picking. `plain` ignores donors; `donor-grouped` never lets a line from a training donor appear in test.

Classification

model	scheme	ROC-AUC	PR-AUC	balanced acc	sensitivity	specificity	F1	Brier
logistic_l2 <i>(headline)</i>	plain	0.940 $\pm$ 0.012	0.964 $\pm$ 0.011	0.914 $\pm$ 0.013 ★	0.931 $\pm$ 0.016	0.898 $\pm$ 0.023 ★	0.943 $\pm$ 0.009	0.102 $\pm$ 0.006
logistic_l2 <i>(headline)</i>	donor-grouped	0.947 $\pm$ 0.007	0.968 $\pm$ 0.004	0.913 $\pm$ 0.013	0.940 $\pm$ 0.007	0.886 $\pm$ 0.022	0.945 $\pm$ 0.007 ★	0.097 $\pm$ 0.008
logistic_l1 <i>(secondary)</i>	plain	0.948 $\pm$ 0.011	0.968 $\pm$ 0.012	0.904 $\pm$ 0.019	0.933 $\pm$ 0.005	0.874 $\pm$ 0.037	0.939 $\pm$ 0.009	0.077 $\pm$ 0.007
logistic_l1 <i>(secondary)</i>	donor-grouped	0.951 $\pm$ 0.008 ★	0.970 $\pm$ 0.009 ★	0.912 $\pm$ 0.013	0.942 $\pm$ 0.011 ★	0.883 $\pm$ 0.021	0.945 $\pm$ 0.008 ★	0.075 $\pm$ 0.004 ★
trivial baseline	—	0.500	0.696	0.500	1.000	0.000	0.821	0.212

Regression

model	scheme	R ²	MAE	RMSE	out-of-range
ridge <i>(headline)</i>	plain	0.676 $\pm$ 0.013	0.134 $\pm$ 0.003 ★	0.170 $\pm$ 0.003	0.050 $\pm$ 0.007
ridge <i>(headline)</i>	donor-grouped	0.653 $\pm$ 0.021	0.138 $\pm$ 0.004	0.175 $\pm$ 0.005	0.043 $\pm$ 0.005 ★
lasso <i>(secondary)</i>	plain	0.680 $\pm$ 0.016 ★	0.134 $\pm$ 0.003 ★	0.169 $\pm$ 0.004 ★	0.049 $\pm$ 0.008
lasso <i>(secondary)</i>	donor-grouped	0.668 $\pm$ 0.015	0.135 $\pm$ 0.003	0.172 $\pm$ 0.004	0.046 $\pm$ 0.004
predict the mean	—	0.000	0.266	0.298	0.000

★ = best in that column among the four fitted rows (baselines excluded). **Bold** marks the pre-registered headline row, which is a separate thing from being best. Higher is better everywhere except **Brier**, **MAE**, **RMSE** and **out-of-range**, where lower is better — so the star follows the minimum in those columns.

Baseline row = what a trivial model scores: always predicting "success" for classification (or random ranking, for the two AUCs), and predicting the training mean for regression. **Sensitivity 1.000 and F1 0.821 from that baseline are why neither is quoted alone** — with 70% successes, guessing "success" every time already looks respectable on both.

The PCA fitting population barely matters. Every figure here uses the baseline basis. Refitting the whole pipeline on cells from the 138 study lines only moves the donor-grouped results by +0.014 R² (ridge), +0.004 (lasso), +0.001 AUC (L2), −0.001 (L1) — all inside about one SD. See §3.

Best model: L2-penalised logistic regression on the binary task. The L1 variants score marginally higher but the difference is well inside one SD, and they were not pre-registered.

Classification substantially outperforms regression, and this is a property of the outcome, not the models.

`diff_efficiency` is bimodal — 42 failures in 0–0.185, 96 successes in 0.221–0.92, with a real gap at the threshold.

Splitting predictions by true clump: within-success R^2 is only ~ 0.17 – 0.22 and within-failure R^2 is strongly negative (≈ -14).

The headline R^2 mostly reflects correctly separating the two clumps, not fine-grained precision. Use this model to call success/failure; do not use its regression output as a precise efficiency estimate.

Robustness across CV schemes (nested, ridge / `logistic_l2`):

scheme	regression R^2	classification ROC-AUC
LOCO (leave-one-line-out)	0.682	0.942
plain 5-fold	0.676	0.940
LODO (leave-one-donor-out)	0.665	0.963
donor-grouped 5-fold	0.653	0.947

Donor leakage is real but small ($\sim 0.012 R^2$), consistent across two independent contrasts (plain→donor-grouped, LOCO→LODO). Classification is essentially flat across schemes.

2. Which features are informative

Assessed five ways — univariate association, standardised coefficient, leave-one-covariate-out, permutation, and SHAP — so that agreement or disagreement between measures is itself informative.

`phat_NB`, the day-11 neuroblast fraction, is the standout feature.

- Strongest univariate association of anything measured: Spearman $\rho = -0.73$ (both tasks). More neuroblasts at D11 → worse D52 yield.
- Largest single LOCO contribution for classification (+0.051 AUC).
- Largest SHAP value for classification (1.19 vs PC2's 0.51).
- **By far the cleanest technically:** pool $\eta^2 = 0.04$, the only "Low" feature in either table.

Dropping all cell-type proportions costs **+0.064 AUC** — the largest single effect in either table — but costs regression essentially nothing (-0.0007). On the restricted basis, the modal nested `logistic_l1` model selects **k=0 PCs** — proportions alone — and still reaches AUC 0.950.

PC1 is a proliferation axis and is genuinely predictive ($\rho = +0.563$). Quantitatively enriched for the Tirosh/Seurat G2/M set (30 of the top 50 positive loadings; Mann-Whitney $p = 2.3 \times 10^{-26}$), and it tracks the *proliferating* FPP fraction at $\rho = +0.798$. More proliferative D11 cultures differentiate better.

LOCO and permutation disagree for the regression proportions, and the disagreement is the finding: permutation $\Delta = 0.035$ (the fitted model does use them) but LOCO $\Delta \approx 0$ (refitting without them costs nothing) — the signature of information that is redundantly available in the PCs. For classification both measures agree the proportions are essential. Running only one measure would have misled in either direction.

3. Technical confounders

Pool identity explains a large share of the variance in most PC features (η^2 of the line-level value on pool):

feature	pool η^2	
phat_NB	0.04	clean
PC4	0.45	moderate
PC1	0.49	moderate
PC3	0.50	moderate
PC2	0.76	high — and the single most predictive PC
PC5	0.79	high
PC6 / PC9 / PC8	0.91 / 0.95 / 0.96	near-pure batch

Two consequences:

1. **PC2 is model-predictive but technically entangled.** It has the largest coefficient, LOCO delta and permutation delta of any PC for regression, while $\sim 76\%$ of its line-level variance is pool-associated. The correct statement is "*PC2 improves prediction under the observed design*", not "*PC2 is biologically predictive*". Its contribution may not transfer to new pools or altered processing.
2. **The excluded PCs are excluded for good reason.** PC6/PC8/PC9 are nearly pure batch signal; the tuned k never reaches them.

Other technical decisions, frozen from EDA before any modelling:

- **pool11 excluded from every PCA/HVG fit** ($\approx 1,900$ mean UMI/cell vs 10,000–18,000 elsewhere) but its cells are still *projected*, so no line is lost from train or test.
- **Fitting population matters.** Restricting the PCA fit to the study population (dropping $\sim 25,400$ cells from 39 lines outside the 138) improved regression R^2 by $+0.014$ and roughly **halved** its across-repeat SD ($0.021 \rightarrow 0.012$). Classification was unaffected.

4. Additional findings worth recording

The PC2–PC3 plane is stable; the individual axes are not. PC2 and PC3 are near-tied in explained variance ($0.0265 / 0.0259$) and *reorder* between PCA fitting variants: baseline PC2 $\leftrightarrow$ qualifying-only PC3 ($|r| = 0.985$), while qualifying-only PC2 is a different component ($|r| = 0.564$). But the *plane* they span is essentially identical (principal angles 6.0° and 1.9°). **"PCn" is not a stable label; align components by correlation before interpreting.** PC1 is the exception — it replicates at $r = +0.9995$ with 25/25 top-gene overlap.

Hyperparameter selection is unstable at $n=138$. Across 50 outer folds the modal nested configuration is chosen only 9–10 times, spread over 13–19 distinct configurations, with C spanning four orders of magnitude. No single configuration is "the" model — which is why the headline is pre-registered and a one-SE rule is used, and why any single-config coefficient table is one draw from a wide distribution.

5. Literature assessment: recapitulated vs. new

The single most important comparison: **Jerber et al. predicted differentiation efficiency from iPSC-stage bulk RNA-seq, before differentiation began — not from day-11 scRNA-seq.** The D11 $\rightarrow$ D52 prediction here is therefore a different analysis on the same data, not a reproduction of theirs.

#	Finding	Status	Basis
1	D11 scRNA-seq features predict D52 efficiency, ROC-AUC 0.947	New	Authors predicted from iPSC-stage bulk RNA-seq; they did not build a D11 → D52 predictor
2	100% precision at 65% recall for detecting poor differentiation	Extends prior result	Authors reported 100% precision at 35% recall from iPSC bulk under LOOCV. Ours is $\sim 2\times$ the recall under a <i>stricter</i> (donor-grouped) scheme — see caveat below
3	D11 neuroblast fraction predicts <i>worse</i> D52 outcome ($\rho = -0.73$)	Recapitulates, and sharpens	Authors noted their poor-differentiation-associated "cluster 2" correlated with D11 neuroblast proportion — an indirect observation. Here <code>phat_NB</code> is the single strongest direct predictor and the most technically clean feature
4	Proliferation (PC1) associated with better differentiation	Consistent with established biology; new in this system	Cell-cycle state governs fate propensity in hPSCs (Pauklin & Vallier 2013); cell-cycle lengthening precedes neurogenesis. Not previously quantified as a D11 → D52 predictor here
5	PC3 = neuronal differentiation vs. S-phase/replication	Recapitulates textbook biology	Cell-cycle exit accompanying neurogenesis is long established. Best read as a <i>positive control</i> that the PCA captures real biology
6	Classification » regression because the outcome is bimodal	New (methodological)	Follows from the authors' own 0.2 threshold but is not analysed by them
7	D11 expression PCs are strongly pool-confounded (η^2 up to 0.96)	New; complements the authors	They showed the <i>outcome</i> is not associated with lines-per-pool ($R^2 = 0.04$, $P = 0.46$) and that line effects dominate. That is about the outcome; this is about the <i>features</i> , where pool structure is large
8	Donor leakage inflates R^2 by only ~ 0.012	New (methodological)	Authors used LOOCV without donor grouping; this quantifies what that costs
9	Hyperparameter selection unstable; PC2/PC3 rotate between fits	New (methodological)	Caution for anyone interpreting a single PCA solution from $n \approx 138$

The important caveat on finding #2

It is **not** a like-for-like "we beat them" comparison, and the practical implication runs the other way:

- Their predictor uses **iPSC-stage** expression, available *before* committing to a differentiation run — the practically useful screening point.
- Ours requires **11 days of culture** before it can say anything.

So theirs answers "*is this line worth differentiating?*"; ours answers "*is this run going to work?*" — a different, later, cheaper-to-abort question. The gain in accuracy is real but is bought with 11 days.

6. What would strengthen this most

1. **Leave-one-pool-out validation.** The decisive test of whether PC2's contribution is line biology or batch. Nothing here separates them.
2. **Within-pool permutation importance** instead of global — global permutation can inflate importance by creating PC×pool combinations absent from the data.
3. **The D30 → D52 baseline**, to establish how much of the D11 signal is simply "differentiation is already visibly on track".
4. **Direct per-cell program scoring** (S-phase, G2/M, neuronal) verified *within* pools, rather than enrichment among extreme loadings.

7. What this does not show

- No causal claim. Nothing here shows that increasing D11 proliferation, or suppressing neuroblast formation, would improve D52 yield.
- No claim that individual top-loading genes are drivers or targets.
- No demonstrated transfer to new pools, protocols or laboratories.

- The regression model does not predict fine-grained efficiency (§1).
-

Sources

1. **Jerber J, Seaton DD, Cuomo ASE, et al. (2021)** "Population-scale single-cell RNA-seq profiling across dopaminergic neuron differentiation." *Nature Genetics* 53:304–312. <https://www.nature.com/articles/s41588-021-00801-6> · <https://pmc.ncbi.nlm.nih.gov/articles/PMC7610897/> — source of the data, the 0.2 threshold, the iPSC-stage predictor (100% precision at 35% recall), the cluster-2/neuroblast observation, and the lines-per-pool null result.
2. **Tirosh I, Izar B, Prakadan SM, et al. (2016)** "Dissecting the multicellular ecosystem of metastatic melanoma by single-cell RNA-seq." *Science* 352(6282):189–196. <https://www.science.org/doi/10.1126/science.aad0501> — G1/S and G2/M gene sets used for the PC1 enrichment test.
3. **Seurat** [cc.genes](https://satijalab.org/seurat/reference/cc.genes) <https://satijalab.org/seurat/reference/cc.genes> — the exact gene lists matched against.
4. **Whitfield ML, Sherlock G, Saldanha AJ, et al. (2002)** "Identification of genes periodically expressed in the human cell cycle and their expression in tumors." *Mol Biol Cell* 13(6):1977–2000. <https://pubmed.ncbi.nlm.nih.gov/12058064/> — foundational human cell-cycle periodicity map underlying the G2/M gene identities.
5. **Pauklin S, Vallier L (2013)** "The cell-cycle state of stem cells determines cell fate propensity." *Cell* 155(1):135–147. <https://pmc.ncbi.nlm.nih.gov/articles/PMC3898746/> — establishes that cell-cycle state governs differentiation propensity in hPSCs; the prior for finding #4.
6. **Hardwick LJA, Ali FR, Azzarelli R, Philpott A (2014)** "Cell cycle regulation of proliferation versus differentiation in the central nervous system." *Cell Tissue Research* 359(1):187–200. <https://pmc.ncbi.nlm.nih.gov/articles/PMC4284380/> — G1 lengthening and cell-cycle exit preceding neuronal differentiation; the prior for finding #5.
7. **Luecken MD, Theis FJ (2019)** "Current best practices in single-cell RNA-seq analysis: a tutorial." *Mol Syst Biol* 15(6):e8746. <https://doi.org/10.15252/msb.20188746> — QC covariates and technical confounders; basis for the §3 caveats.
8. **Huang Y, McCarthy DJ, Stegle O (2019)** "Vireo: Bayesian demultiplexing of pooled single-cell RNA-seq data without genotype reference." *Genome Biology* 20:273. <https://pmc.ncbi.nlm.nih.gov/articles/PMC6909514/> — pooled/multiplexed design context: pooling controls *within-pool* batch variation but leaves pool-to-pool structure, which is what §3 measures.

Modeling methods and full results

source: `modeling/README.md`

The complete method record: CV design, metrics, feature importance, every correction found by audit, and D30 readiness.

D11 -> D52 modeling: CV/modeling plan

Project-level findings summary with literature context: [FINDINGS.md](#)

Predicting D52 differentiation efficiency from D11 features (138 cell lines, from `metadata_eda/cohort/qualifying_cell_line_pool_min10_per_timepoint.csv` and `metadata_eda/cohort/d52_diff_efficiency_label.csv`). Designed to generalize to a D30 -> D52 baseline model later (see Architecture).

Layout

```
modeling/
  *.py          the package: folds, features, models, harness,
                run_* entry points, feature_importance
  tests/        pytest suite (30 fast + 1 slow integration test)
  fold_data/    per-fold feature tables and fold assignments
                (`_full` = baseline basis, `_qualonly` = restricted-fit variant)
  results/      CV grids, nested per-fold selections, feature-importance tables
  docs/         PCA_interpretation.md, feature_importance_plan.md,
                pool_correction_investigation.md, plus two standalone
                HTML explainers (open in a browser):
                eta_squared_and_r2.html -- what the pool eta^2 column means
                roc_auc_and_pr_auc.html -- ROC/PR, and the 100%-precision point
  plots/        diagnostic figures
  archive/      superseded first-pass artifacts, kept for provenance only --
                nothing reads these
```

Writers create their own subdirectory, so a fresh clone or a redirected `out_dir` works without manual `mkdir`.

First-pass prototype scope (decided while implementing)

Timed a single fold's PCA fit empirically: `compute_gene_stats` 11.2s + `extract_hvg_matrix` 7.6s + `run_pca` 29.1s (k=10) / 23.5s (k=5) = 48-60s per fold. The two streaming passes don't depend on `k` at all, and the PCA step itself only saves ~12% going from k=10 to k=5 (randomized SVD cost scales mostly with matrix size -- 250K cells x 2000 HVGs -- not much with target rank in this range). This changed two decisions:

- **`n_pcs` stays at 10, not reduced to 5** -- reducing it barely helps runtime (~12% off the PCA step only) so isn't worth losing PCs 6-10 for a first pass.
- **Repeats = 5, not 10 or 3** -- 3 is too thin to see a real distribution of results (the stated goal); 10 is the better final number eventually, but for a first-pass prototype the marginal precision isn't worth the extra ~50 min, especially once we noticed LOCO/LODO (below) is the actual dominant cost, not repeats.
- **This first pass runs `plain` + `donor_grouped` only; LOCO and LODO are deferred**, not dropped. Total folds = `2 x 5 x n_repeats (plain + donor_grouped) + 138 (loco) + 20 (lodo)` -- LOCO alone (138 folds) is larger than even the full 10-repeat plain+donor_grouped schemes combined (100 folds), so it dominates total cost far more than the repeats count

does. Validating the harness end-to-end on the cheaper schemes first, before paying for LOCO/LODO's ~2.4 hr, is the more sensible order for troubleshooting.

- Estimated cost for this first pass: 50 folds (5 x 5 plain + 5 x 5 donor_grouped), k=10, ~48-50s/fold once the process is warm ≈ ~40-45 min, run as a single background job (avoids paying Python/import startup cost per fold).

Full results (258 folds: plain 50, donor_grouped 50, LOCO 138, LODO 20)

Headline (donor_grouped, nested CV, **no pool-correction** -- see "Pool-correction dropped" below):

The pre-registration named a specific model per task (**ridge / logistic_l2**), so those are the headline numbers. The L1 variants scored better, but quoting them as "the" headline would be exactly the cherry-picking the pre-registration exists to prevent -- they are reported below as secondary.

task	model	key metrics
regression (headline)	ridge	MAE=0.138, RMSE=0.175, R2=0.653 +/- 0.021
regression (secondary)	lasso	MAE=0.135, RMSE=0.172, R2=0.668 +/- 0.015
classification (headline)	logistic_l2	ROC-AUC=0.947 +/- 0.007 , PR-AUC=0.968, balanced_acc=0.913
classification (secondary)	logistic_l1	ROC-AUC=0.951 +/- 0.008, PR-AUC=0.970, balanced_acc=0.912

Full metric set for every model and scheme is in "Latest results" at the bottom; what each metric means and why it is in the set is under "Metrics".

D11 features are clearly predictive of D52 differentiation efficiency, even under the strict donor-grouped (never-seen-donor) test. Going from the 5-repeat first pass to the full 10-repeat + LOCO/LODO run barely moved these numbers (secondary lasso R2 0.667->0.668, secondary L1 AUC 0.949->0.951) -- the estimates were already stable.

All four schemes, nested CV:

scheme	regression R2 (lasso / ridge)	classification ROC-AUC (L1 / L2)
LOCO (leave-one-line-out)	0.691 / 0.683	0.931 / 0.939
plain 5-fold	0.680 / 0.677	0.947 / 0.945
LODO (leave-one-donor-out)	0.679 / 0.665	0.953 / 0.958
donor_grouped 5-fold	0.668 / 0.653	0.951 / 0.943

Donor-leakage effect is real but small (~0.012 R2), and consistent across two independent comparisons: plain vs. donor_grouped (0.680->0.668) and LOCO vs. LODO (0.691->0.679) give the same gap. The scheme ordering is mechanistically sensible -- LOCO is most optimistic (137/138 lines in training AND siblings allowed), donor_grouped strictest (siblings excluded AND ~28 lines held out), LODO in between (prevents donor leakage but still trains on ~131-137 lines). **Classification is essentially flat across all four schemes (0.93-0.96)** -- no meaningful donor-leakage penalty at all.

Caveat on LOCO: its metrics pool all 138 single-line fold predictions (a one-line fold can't support per-fold metrics), so it has no variance estimate and isn't perfectly comparable to the repeat-averaged schemes.

Caveat on the regression R2 (checked, see docs/pool_correction_investigation.md "Follow-up"): mostly reflects correctly separating success from failure, not fine-grained precision. The outcome is bimodal (42 failures at 0-0.185, 96 successes at 0.221~0.92, real gap at the threshold). Splitting predictions by true clump: within-success R2 is modest but real (~0.17-0.22); within-failure R2 is strongly negative (~-14, i.e. far worse than a trivial baseline -- absolute error ~0.135 is nearly the whole failure clump's true range). Trust this model for success/failure classification; don't trust the regression output as a precise efficiency estimate, especially for predicted failures.

Visual confirmation in plots/plot_regression_diagnostics.png / plots/plot_classification_diagnostics.png (predicted-vs-true and residual scatter for regression; ROC curve, confusion matrix, and predicted- probability histogram for classification, all donor_grouped/nested/ repeat 0). Two things visible there that the numbers alone didn't show: - Classification's predicted probabilities are almost perfectly bimodal (failures near 0, successes near 1, almost nothing in between) -- directly mirrors the gap in the true label and is why AUC~0.94 is achievable even though the regression signal is much weaker. - Within the success clump, regression residuals show a **systematic bias, not just noise**: over-predicts around

true~0.3-0.4, under- predicts around true~0.8-0.9 -- the model compresses predictions toward the middle of the success range (shrinkage/regression-to-the- mean) rather than tracking the full spread.

Which features matter (feature importance)

`feature_importance.py` -> `results/feature_importance_table_{task}_{model}.csv`. Design and caveats in `docs/feature_importance_plan.md`. The single configuration these tables are fitted at comes from the **flat** grid, not nested -- see "Flat vs nested: which is used for what" for why. Results below are on the full 258-fold data; they were essentially unchanged from the 5-repeat first pass (selection frequencies firmed up slightly, LOCO deltas moved by <0.005), so they look stable.

These tables are the BASELINE basis only. For the current numbers on both bases, including the qualifying-only variant, see "**Latest results**" at the bottom of this file -- that section is authoritative. The narrative and interpretation here still hold; only read the specific figures from the bottom table.

What the columns mean

`phat` is **p-hat**, statistics notation for an estimated proportion. `phat_NB` = fraction of that line's D11 cells labelled neuroblast, computed per `(cell_line, pool)` as `n_of_that_type / n_total`, then averaged pool-then-line. The three sum to exactly 1, which is why only two enter a fit and `phat_P_FPP` reads `reference`.

column	how it is computed	how to read it	example (logistic_l1, k=2)
Univariate rho	Spearman rank correlation of feature vs outcome over the 138 lines. No model, no folds, no other features.	-1..+1. Sign = direction, magnitude = monotonic association. Marginal, so blind to confounding.	<code>phat_NB</code> -0.733: more neuroblasts -> lower efficiency
Coef	Standardized coefficient from ONE fit on all 138 lines at the winning config; features scaled to mean 0 / SD 1 first.	Change in outcome per 1 SD of the feature; log-odds for classification. Standardization is what makes proportions (SD ~0.05) and PCs (SD ~1-9) comparable.	<code>phat_NB</code> -1.535: +1 SD of NB lowers log-odds of success by 1.53
Sel. freq	Fraction of CV folds where the coefficient came back non-zero.	0..1 stability -- does it survive refitting on other subsets? Only meaningful for L1 ; ridge/L2 never zero anything, so their column is uniformly 1.00.	<code>phat_NB</code> 0.90 kept; <code>phat_FPP</code> 0.00 always dropped
LOCO delta	Refit without that feature, same folds and config; paired per fold, <code>score_with - score_without</code> , then averaged.	Positive = removing it hurt = it helps. In headline-metric units. Because the model is refit, a real but redundant feature scores ~0.	<code>phat_NB</code> +0.051 AUC
Perm delta	Take the already-fitted model, shuffle that column in the test matrix, re-score. Paired per fold. Proportions shuffled as a block.	Positive = this model relies on it. No refit, so it measures dependence, not irreplaceability.	<code>phat_FPP</code> 0.277 -- yet its LOCO is 0.0
SHAP	<code>mean abs(coef_j * z_ij)</code> over the 138 lines, <code>z</code> = standardized value.	Average magnitude of the feature's push on one prediction.	<code>phat_NB</code> 1.190 vs PC2 0.514
Technical covariate	η^2 of the feature on pool identity: $(SS_{total} - SS_{within})/SS_{total}$.	0..1 = fraction of the feature's variance explained by which pool a line came from. Says nothing about the outcome -- it flags the feature. >=0.5 High, >=0.15 Moderate, else Low. Compare against the chance floor below, not against 0.	<code>phat_NB</code> 0.04 clean; PC2 0.76 mostly batch

η^2 has a chance floor -- read it against that, not against 0. Fitting 10 pools (9 dummies) to 159 rows explains some variance even for a feature with no pool structure at all. Permuting a random feature 2,000 times over the real pool labels gives:

```
null eta^2: mean 0.057, 95th percentile 0.105
            (chance level ~ (n_pools-1)/(n-1) = 0.057)
```

So the "Low" band (<0.15) is only about 2x chance. Against that null, every PC and both of `phat_FPP` / `phat_P_FPP` sit far above chance ($p < 0.0005$), while **`phat_NB` at 0.037 is at or below what a random feature scores** -- a stronger statement than "Low": there is no detectable pool structure in it at all.

`sqrt(eta^2)` is the correlation ratio, comparable in scale to `abs(r)`: PC2's 0.764 -> 0.87, `phat_NB`'s 0.037 -> 0.19.

Two honest caveats:

- **SHAP is not a fifth independent measure here.** For a linear model on standardized features $\text{mean } \text{abs}(z) \sim 0.8$, so $\text{SHAP} \sim 0.8 \times \text{abs}(\text{coef})$ (observed 0.78 and 0.79). It is a consistency check that respects the feature's actual distribution, not corroborating evidence.
- **Coefficient signs are multivariate and can flip.** In the regression table `phat_NB` has $\rho = -0.73$ but coefficient **+0.067** -- a suppression effect, with PC2/PC3 absorbing the shared signal. Never read a coefficient sign in isolation.

Why five measures rather than one: they disagree, and the disagreement is the finding. LOCO ~ 0 with a large permutation delta means real but redundant information; both large means irreplaceable. `phat_NB` is the standout precisely because every measure agrees *and* it is the only technically clean feature in either table.

What the "one-SE rule" is

A standard way to pick a hyperparameter config that avoids chasing noise. Instead of taking the raw best-scoring config (the argmax across all 55 `(k, regularization)` combos), you:

1. Find the best mean cross-validated score.
2. Take every config whose score is within **one standard error** of that best score -- i.e. everything statistically indistinguishable from the winner given how noisy the estimate is.
3. Among those, pick the **simplest** one.

Rationale: with $n=138$ and 55 candidates, the literal argmax is often a noise peak -- some config got lucky on these particular folds. Anything within 1 SE of it is, on the evidence, just as good, so preferring the simplest of them gives a more robust and more interpretable model at no real cost in performance. "Simplest" here is defined in advance (not after seeing results) as: fewest PCs first, then strongest regularization (larger `alpha` for ridge/lasso; smaller `C` for logistic, since `C` is inverse regularization strength).

The one-SE rule is applied ONLY in flat CV, never inside nested CV. `select_winning_config_one_se` lives in `feature_importance.py` and is called only from there, on the flat grid. Nested CV's inner loop (`harness.run_nested_cv`) takes the **raw argmax** of the mean inner score:

```
mean_inner = float(np.mean(inner_scores))
if mean_inner > best_score:                # plain argmax, no tolerance band
    best_score, best_combo = mean_inner, (k, param_combo)
```

This is not a correctness problem -- nested CV stays unbiased either way, because the selection happens inside the outer fold and never sees the outer-test lines. But it is worth knowing for two reasons:

1. **The inner loop is the noisier of the two selections**, not the cleaner one. It ranks the same 55 candidates using only 3 inner folds of ~ 36 validation lines each, versus flat's 10 repeats over all 138. If argmax on 55 candidates is a noise peak anywhere, it is there.
2. **It partly explains the hyperparameter instability documented below** -- the modal nested config winning only ~ 9 -10 of 50 outer folds is exactly what raw argmax over 55 near-tied candidates on small inner folds produces. A one-SE (or any tolerance) rule in the inner loop would concentrate those selections considerably.

Applying it there would change reported performance, so it has not been done on a whim; it is a live option rather than an oversight.

Implementation note: the tolerance band is $\text{SD_across_repeats} / \sqrt{n_repeats}$, with `n_repeats` read from the fold-assignment data. (An earlier version used the raw SD, which is $\sim 3\times$ too wide at 10 repeats -- see "Corrections from the audit". Fixed; selections happened not to change.) Effect here: classification went from the raw argmax's $k=4$ to $k=2$ -- a materially simpler model for statistically indistinguishable performance.

Regression (lasso, one-SE-selected $k=5$):

Feature	Univariate ρ	Coef	Sel. freq	LOCO Δ	Perm Δ	SHAP	Technical covariate
phat_FPP	0.19	-0.045	0.88	-0.0015	0.035 (grp)	0.033	High ($\eta^2=0.76$)
phat_NB	-0.73	+0.067	1.00	+0.0017	0.035 (grp)	0.052	Low ($\eta^2=0.04$)
phat_P_FPP	0.34	reference	--	-0.0007 (grp)	0.035 (grp)	--	High (0.65)
PC1	0.56	0.004	0.80	-0.0024	0.005	0.003	Moderate (0.49)
PC2	0.67	+0.312	1.00	+0.0186	0.176	0.247	High ($\eta^2=0.76$)
PC3	-0.22	-0.251	1.00	+0.0141	0.146	0.189	Moderate (0.50)
PC4	0.53	0.044	1.00	-0.0009	0.015	0.035	Moderate (0.45)
PC5	-0.02	-0.052	1.00	+0.0016	0.009	0.040	High (0.79)

Classification (logistic_11, one-SE-selected k=2 -- notably simpler than the raw argmax's k=4, the one-SE rule working as intended):

Feature	Univariate ρ	Coef	Sel. freq	LOCO Δ	Perm Δ	SHAP	Technical covariate
phat_FPP	0.20	0.0 (regularized out)	0.00	0.0	0.277 (grp)	0.0	High (0.76)
phat_NB	-0.73	-1.535	0.90	+0.051	0.277 (grp)	1.190	Low ($\eta^2=0.04$)
phat_P_FPP	0.41	reference	--	+0.064 (grp)	0.277 (grp)	--	High (0.65)
PC1	0.58	0.0 (regularized out)	0.10	0.0	0.000	0.0	Moderate (0.49)
PC2	0.59	+0.648	0.82	+0.004	0.080	0.514	High (0.76)

("grp" = grouped: permutation always shuffles the 3 proportions as one block, since permuting one alone implies an impossible third coordinate.)

LOCO vs. permutation disagree for the regression proportions, and the disagreement is the finding: permutation $\Delta=0.035$ (the fitted model does rely on them) but LOCO $\Delta\approx 0$ (refitting without them costs nothing). That's the correlated-feature signature -- the proportions carry real signal, but it's redundantly available in the PCs, so a refit compensates. For classification both measures agree they're essential (perm 0.277, LOCO 0.064, both largest in the table), meaning there the proportions carry something the PCs don't replicate. Running only one of the two measures would have told a misleading story either way.

Three findings: 1. **phat_NB is the standout trustworthy feature** -- strongest univariate signal of anything ($\rho=-0.73$), near-always selected, largest individual LOCO contribution for classification, and by far the cleanest technically ($\eta^2=0.04$, the only "Low" feature in either table). The one result to lean on biologically. 2. **PC2 is the strongest PC for regression but is heavily pool-confounded** ($\eta^2=0.76$) -- largest coefficient, permutation and LOCO delta for regression, but its predictive power may be substantially technical rather than biological. Treat PC2-based claims cautiously. (Note: an earlier version of this README claimed PC2 also dominated SHAP; that was a units bug -- see "Corrections from the audit" below. Corrected, phat_NB has the larger SHAP value for classification, 1.19 vs PC2's 0.51.) 3. **Proportions matter greatly for classification, almost not at all for regression** -- dropping both costs +0.064 AUC (largest single effect in either table) but costs regression $\sim$ nothing (-0.0007).

Caveat: phat_NB's univariate correlation is negative (-0.73) but its *regression* coefficient is positive (+0.067) -- a suppression effect from multivariate adjustment (PC2/PC3 absorb the shared signal). Don't read that sign in isolation; the univariate direction and the classification coefficient (-1.53) agree that more NB at D11 -> worse D52 outcome.

Why the tables stop at PC5 (regression) / PC2 (classification)

k is a tuned hyperparameter, so the tables only list features actually *in* the selected model. The flat-CV grid tested k=0..10 exhaustively; larger k didn't score better, so the one-SE rule took the simpler model.

Known design limitation (raised by Codex): we only ever test PC *prefixes* (PC1..k), never arbitrary subsets. PCs are ordered by variance in the *predictors*, not by outcome relevance -- so a predictive PC8 sitting behind noisy PC6/PC7 could be missed, since reaching it requires accepting k=8 and dragging the noise in with it.

Checked this directly -- univariate association (no model involved) for every PC, including the excluded ones:

PC	ρ (regression)	ρ (classification)	pool η^2	In model?
PC1	0.563	0.584	0.490	both
PC2	0.670	0.593	0.764	both
PC3	-0.221	-0.370	0.498	regression only
PC4	0.525	0.477	0.448	regression only
PC5	-0.018	-0.015	0.788	regression only
PC6	-0.101	-0.089	0.909	no
PC7	0.275	0.364	0.531	no
PC8	0.260	0.197	0.963	no
PC9	0.061	-0.147	0.946	no
PC10	-0.203	-0.157	0.461	no

Conclusion: little is being missed. No excluded PC approaches the included ones' univariate strength (best excluded is PC7 at $\rho \approx 0.28$ -0.36 vs. PC2's 0.67). More tellingly, **PC6/PC8/PC9 are nearly pure batch signal** ($\eta^2 = 0.91 / 0.96 / 0.95$ -- PC8's means 96% of its variance is explained by which pool a line came from). Excluding them is a feature, not a loss.

One nuance: **PC5 is included despite ~zero univariate correlation** (-0.018). Since the one-SE rule takes the simplest config within tolerance and still chose k=5 over k=4, PC5 must contribute multivariately despite being marginally useless -- classic suppressor behavior (correlating with noise in the other PCs so the model can cancel it out). Its own LOCO (+0.0016) and permutation (0.0085) deltas are small, consistent with a minor supporting role rather than a driver.

Fitting population: the qualifying-only PCA variant

The D11 HVG/PCA basis was originally fit on every cell except the fold's held-out lines and pool11. That INCLUDES ~25,400 cells from 39 cell lines that are not among the 138 and never appear in any train or test set, plus ~1,100 cells from non-qualifying pools of qualifying lines. Not leakage (the held-out mask applies by `cell_line`, so every cell of a held-out line is dropped regardless of pool), but not the study population either.

Both variants are now built and kept side by side:

	fit cells (one fold)	outputs
baseline	243,066	*_full.csv , unsuffixed tables
qualifying-only	217,211	*_qualonly.csv

Fold assignments are byte-identical between the two runs (verified: 35,604 rows each), so the comparison is like-for-like and any difference is attributable to the fitting population alone.

Result — pre-registered headline (donor_grouped, nested):

	baseline	qualifying-only
regression, ridge, R2	0.6531 +/- 0.0211	0.6672 +/- 0.0124
classification, logistic_l2, ROC-AUC	0.9470 +/- 0.0070	0.9483 +/- 0.0157

Regression improves modestly (+0.014 R2, ~0.67 SD) and — more notably — its across-repeat SD nearly halves. Classification is a wash: AUC up a hair, balanced accuracy down a hair, SD doubled. So the extra 25,400 foreign cells were adding noise to the regression basis rather than helping it.

Do NOT compare the two importance tables row by row

The components REORDER between bases, so "PC2" does not mean the same thing in each. Correlating per-line coordinates across the 138 lines:

	closest match	r	
baseline PC1	qualonly PC1	0.995	same component
baseline PC2	qualonly PC3	0.985	same component
qualonly PC2	baseline PC2	0.564	a different component

Read positionally, the qualonly table looks like the fit restriction cleaned the pool signal out of the strongest PC (eta2 by pool 0.764 -> 0.114). It did not. The pool-associated axis MOVED from position 2 to position 3 and became slightly *more* pool-associated (eta2 0.764 -> 0.832). What rose into position 2 is a genuinely different component that happens to be largely pool-independent (eta2 0.114) and strongly outcome-associated (Spearman rho -0.745 vs binary `success`, -0.809 vs continuous `diff_efficiency`).

Explained variance is near-identical across bases (0.0539/0.0265/0.0259/0.0203 -> 0.0539/0.0268/0.0254/0.0203), i.e. PC2 and PC3 are near-tied, which is exactly the regime where ordering is unstable. **Align components by correlation before interpreting anything biologically.**

Corrections from the audit

After the analysis was complete, an independent Codex correctness audit (implementation-vs-intent, not methodology) checked every stated design intention across `folds.py`, `features.py`, `harness.py`, `feature_importance.py`, and `run_experiment.py`. Most passed. Three real defects were found and fixed; all three are recorded here rather than quietly patched, since two of them changed reported numbers.

1. **SHAP mixed unit systems (changed a conclusion).** Coefficients come from a model fit on standardized features (units: per 1 SD), but they were multiplied by `raw (x - mean)` deviations. Every contribution was therefore scaled by that feature's raw SD, systematically inflating wide-scale features (PCs, raw SD ~1-9) and deflating narrow ones (proportions, raw SD ~0.05) -- exactly the comparison the column exists to make. Fixed to use standardized deviations. **This reversed the classification SHAP ranking:** `phat_NB` went 0.068 -> 1.190 and PC2 went 2.438 -> 0.514, so `phat_NB` now leads rather than trailing PC2 by ~36x. It strengthens the `phat_NB` finding and weakens the PC2 one; the earlier README claim that PC2 dominated SHAP was wrong.
2. **The "one-SE rule" used one SD, not one SE.** ~3x too wide at 10 repeats, so it selected simpler models than a real one-SE rule would. Fixed to SD/sqrt(n_repeats). Verified the tolerance genuinely narrowed (lasso: 5 configs within tolerance -> 1), but the *selections* happened not to change (k=5 regression, k=2 classification both still qualify), so no downstream numbers moved.
3. **`select_headline` didn't filter by model, and the reporting compounded it.** The pre-registration named ridge / logistic_l2, but the function returned all models and the better-scoring non-registered models (lasso R2=0.668, logistic_l1 AUC=0.951) were being quoted as "the" headline -- the exact cherry-picking the pre-registration exists to prevent. Fixed to filter on the pre-registered model; headline numbers are now ridge R2=0.653 and logistic_l2 AUC=0.943, with the L1 variants reported as secondary.

Feature-importance tables are now produced for **all four** models

(`results/feature_importance_table_{task}_{model}.csv`) rather than just the L1 pair, for the same reason. The L1 tables remain the more informative ones (only L1 produces sparsity, so `regularized_to_zero` / `selection_frequency` are meaningful), but both are reported.

Second correctness review (variant plumbing + test pollution)

A second independent Codex review, run specifically on the qualifying-only work, found two more defects. Recorded here because one of them means a previous entry in this README overstated what had been fixed.

All of these share one root cause: **adding a `--suffix` variant required threading "which variant am I?" through several layers, and it was threaded through some but not others.** Every resulting failure produced plausible, non-crashing, wrong output.

1. **Mixed-basis importance tables.** `--suffix` redirected the fold-features input and the output filenames, but `load_full_fit_features()` always read 005's baseline global PCA. So a single `_qualonly` table had fold-level columns (LOCO, permutation, selection frequency) from one basis and full-fit columns (coefficient, SHAP, univariate, technical covariate) from another. The tell was that the variant's `univariate_spearman` column was byte-identical to baseline. Fixed by parameterizing 005 (`--restrict-to-qualifying`, `--suffix`) and threading `pca_csv` through every full-fit consumer.

2. **That fix was incomplete, and was described here as complete.** `compute_shap_like` was still calling `load_full_fit_features()` with no argument, so `shap_mean_abs` stayed on the baseline basis. The edit had been applied with a string replacement whose pattern did not match, and string replacement fails silently. Re-applied with an assertion that the edit landed plus a check that no no-arg call survives.
3. **The test suite was writing into the repo.** `run_all` saves `results/nested_selections_{task}.csv` to the package directory, and `test_run_experiment.py`'s fixture patched its inputs but not that path. Every `pytest` run overwrote the committed `nested_selections_regression.csv` with 32 rows of synthetic fixture data (`k=1`, `alpha=1.0`, and `repeat=1` for LOCO/LODO, which is structurally impossible) — and that corrupted file had been committed. `run_all` now takes an injectable `out_dir`, the fixture redirects it, and a regression test asserts `run_all` never touches committed files.
4. **`_infer_n_repeats` read a hardcoded table.** It always loaded `fold_data/fold_features_D11_full.csv` (falling back silently to `fold_features_D11.csv`), not the table the run was using, so the one-SE tolerance could come from an unrelated experiment. Harmless in fact here — both runs share fold assignments, so `n_repeats` was 10 either way — but nothing enforced that. Now required and passed explicitly; a missing file is a loud error.
5. **`logistic_l1` was not reproducible.** `saga` is a stochastic solver and no `random_state` was set, so coefficients drifted run to run (measured $\max|\text{diff}| \sim 5.7\text{e-}4$), producing spurious 4th-decimal diffs on re-runs. `models.SEED` now pins both logistic factories.

A guard now prevents defect 1 from recurring silently: with `--suffix` set and no explicit basis, the matching suffixed basis is REQUIRED, and its absence is a hard error naming the command that generates it.

D30 readiness — NOT ready (read before starting D30 -> D52)

The earlier claim in "Architecture" that feature extraction is simply "timepoint-parameterized" is too optimistic. What is genuinely reusable: `folds.py` (fold construction is timepoint-independent — same 138 lines, same D52 labels), `models.py`, most of `harness.py`, and `run_experiment.run_all` given a correct feature table.

What must change first, ranked by risk. The loud failures are fine; the SILENT ones are the danger, and they are marked.

1. **Cell-type schema differs and is hardcoded.** D11 has 3 types (`FPP`, `NB`, `P_FPP`); D30 has 7 (`DA`, `Epen1`, `FPP`, `P_FPP`, `Sert`, `U_Neur1`, `U_Neur2`). `harness.ALL_PROPORTION_COLS` / `MODEL_PROPORTION_COLS` hardcode the D11 three. Fails loudly on a missing column, but the compositional reference category must be chosen and documented for 7 types, not patched ad hoc.
2. **SILENT: extraction is hardcoded to D11.** `run_feature_extraction.py` passes "D11" to `load_cell_metadata` and `compute_pca_features_for_fold` and names outputs `fold_features_D11{suffix}.csv`. Changing only the output filename would produce a plausible file named D30 containing D11 features.
3. **SILENT: resume has no provenance.** The incremental-write resume keys only on (`scheme`, `repeat`, `fold`). It does not record timepoint, fitting population, or PCA settings, so reusing a destination after changing any of those silently keeps stale rows. Write a provenance header/sidecar and refuse to resume across a settings change.
4. **SILENT: full-fit inputs are D11-specific.** `005` is hardwired to `DAY11_FILE` and `d11_*` output names; `feature_importance.py` hardcodes `D11_PCA_CSV`, `D11_PROPORTIONS_CSV` and derives the variant basis as `d11_pca_coords_per_line{suffix}.csv`. Passing D30 fold features *unsuffixed* would combine D30 fold-level values with D11 global PCA and D11 proportions. (A suffixed call would fail loudly on the missing `d11_{...}{suffix}` file.) Make these explicit parameters.
5. **`run_variant.py` input is hardcoded** to `fold_features_D11{suffix}`.
6. **Decide whether excluding pool11 is still justified at D30.** It is currently inherited from a D11 sequencing-depth observation without re-examination.

Note the qualifying (`cell_line`, `pool`) combos ARE the same 159 across 138 lines at both timepoints (qualification already requires ≥ 10 cells at D11 and D30 and D52), but the per-combo cell COUNTS are not: zero of 159 combos has identical D11 and D30 counts (D11 range 25-14,640; D30 range 13-13,112; correlation 0.748). Sampling precision therefore differs substantially, which is a further reason D11 proportions must never be reused as D30 features.

What PC1 is: a D11 proliferation axis

Full write-up, loadings tables and literature sources: [modeling/docs/PCA_interpretation.md](#) . Summary below.

`005_d11_pca_features.py` now also writes `metadata_eda/pca/d11_pca_gene_loadings{suffix}.csv` -- all 10 PCs x 2000 HVGs, long format, sorted by `|loading|` within each PC.

PC1 (5.4% of HVG variance, Spearman rho = **+0.563** vs D52 efficiency, so higher PC1 = higher efficiency) is a **cell-cycle / proliferation axis**:

positive loadings (high in high-efficiency lines)	negative loadings
HMGB2, PTTG1, NUSAP1, UBE2C, CENPE, CKS2, TOP2A, PLK1, CCNB1/CCNB2, KPNA2, CDC20, BIRC5, CKS1B, AURKA/AURKB, CCNA2, TPX2, CDK1, KIF2C, SMC4, CDKN3, MKI67	RPL12, RPL10, RPS3, RPS12, RPS28, RPL37A (ribosomal proteins); EIF3E, EIF3L, EIF4A2 (translation initiation); GAPDH, COX7C, UQCRCB, TOMM7, ETFB (housekeeping/OXPHOS); SNHG8, MIAT, EPB41L4A-AS1, APOE, CCND2, SLC2A1

The positive side is a textbook G2/M signature. So lines whose D11 cultures are **more proliferative differentiate better by D52**. This is consistent with the line-level correlations: PC1 tracks `phat_P_FPP` (proliferating FPP) at rho = **+0.798** and is negatively correlated with `phat_NB` (rho = -0.552), the neuroblast fraction that independently predicts *worse* outcome.

Top 25 genes carry 29.6% of PC1's total weight (loadings are unit-norm); median `|loading|` is 0.0012, so PC1 is dominated by a compact, coherent gene set rather than being diffuse.

Replicates across bases: PC1 loadings correlate $r = +0.9995$ between the baseline and qualifying-only bases, with 25/25 top-gene overlap. Unlike PC2/PC3, PC1 is stable and safe to compare across the two.

Caveats: - **Loading sign is arbitrary in general**. It is interpretable here only because PC1 was oriented against the outcome ($\text{rho} > 0$). Do not carry the sign convention to other PCs without re-checking. - Loadings are weights on **standardized** expression (per 1 SD of that gene, after the `+/- SCALE_CLIP clip`), so they are comparable across genes of different absolute expression -- but they are not fold-changes. - **PC1 is moderately pool-associated ($\text{eta}^2 = 0.49$)**. A proliferation-vs-ribosomal/housekeeping contrast is also the classic shape of a library-size / cell-quality technical axis, so some of this variance is likely technical. The biological reading is plausible and matches the `phat_P_FPP` correlation, but PC1 is not as technically clean as `phat_NB` ($\text{eta}^2 = 0.04$).

Pool-correction dropped

Investigated (with an independent Codex review) why pool-correction cut regression R^2 by ~ 0.10 - 0.12 but barely touched classification AUC. No single settled explanation emerged -- see [docs/pool_correction_investigation.md](#) for the concise summary (my reasoning, Codex's oracle-test refutation of my leading theory, its alternative explanation, and two real implementation issues found along the way). Decision: drop pool-correction from the active pipeline rather than resolve this now -- it added significant complexity for an unclear net benefit at this prototype stage. `features.py` / `harness.py` still support it if revisited.

Setup

- 138 lines, one row each. Regression (`diff_efficiency`) + classification (≥ 0.2), same features/folds for both.
- Features: 3 D11 proportions (always in) + up to 10 PCs, $k=0..10$ tuned by truncating one PCA fit (no refit per k).
- Models, full parallel arms: Ridge + Lasso (regression); L2 + L1 logistic (classification).

Hyperparameters: 55 configurations per model

Two are tuned, jointly: k (11 values) x regularisation strength (5 values).

hyperparameter	values	n
k -- number of PCs	0, 1, 2, ... 10	11
regularisation strength	see per-model grid below	5

$k=0$ means proportions only. The 2 modelled proportions are always in; k controls how many PCs join them. PCs are hierarchical, so k truncates columns from one PCA fit -- no refit per k .

model	task	parameter	values
ridge	regression	alpha	0.01, 0.1, 1, 10, 100
lasso	regression	alpha	0.001, 0.01, 0.1, 1, 10
logistic_l2	classification	C	0.01, 0.1, 1, 10, 100
logistic_l1	classification	C	0.01, 0.1, 1, 10, 100

C is sklearn's inverse regularisation strength -- the logistic equivalent of `alpha`, running the opposite direction ($C \sim 1/\alpha$):

- `alpha up` -> more regularisation -> coefficients shrink
- `C up` -> less regularisation -> coefficients grow freely

That inversion is why "simplest" in the one-SE rule means *larger alpha*, *smaller C*: both mean more heavily regularised.

Lasso's `alpha` grid sits one decade lower (0.001-10) because L1 shrinks coefficients to exactly zero, so it needs smaller values before it stops zeroing out everything.

CV — both variants run, neither "primary"

- **Plain split**: ordinary split over the 138 rows (grouping by `cell_line` is a no-op here — each row is already one whole line).
- **Donor-grouped split**: lines sharing a donor forced into the same fold (136/138 lines have a donor-sibling; one donor has 18 lines). See "What donor_grouped actually does" below.
- **LOCO + LODO**: full leave-one-line-out and leave-one-donor-out runs, as comparability checks vs. the paper's (non-donor-grouped) LOOCV.
- **Flat CV + nested CV**: both run (flat = grid search, report best mean CV score; nested = outer loop for honest estimate, inner loop tunes `k` /regularization). Inner tuning metric: PR-AUC (classification), MAE (regression). They are used for **different jobs** -- see below.
- **Repeated K-fold**: report mean $\pm$ SD **per repeat** — never pool predictions across repeats (double-counts each line); pooling *within* one repeat is fine.
- **Weighting**: equal weight per line (not per donor).

How the 138 lines are split, by scheme

scheme	folds per repeat	outer test set	repeated?	final result
plain 5-fold	5	27-28 lines	10x	10 pooled metrics -> mean +/- SD
donor-grouped 5-fold	5	24-33 lines, all from held-out donors	10x	10 pooled metrics -> mean +/- SD
LOCO	138	exactly 1 line	no	138 predictions -> one pooled metric, no SD
LODO	20	1-18 lines (one donor's entire set)	no	138 predictions -> one pooled metric, no SD

In every scheme a repeat's folds partition all 138 lines, so each line is predicted exactly once per repeat and the pooled metric is always computed on $n=138$. What differs is *how* the held-out set is chosen and how many times the split is redrawn.

Three things this makes visible:

- **Donor-grouped folds are uneven (24-33), plain folds are not (27-28)**. Grouping constrains whole donors into a fold, so sizes cannot be balanced exactly. The donor with 18 lines forces one fold to be large.
- **LOCO and LODO are not repeated**, because they are exhaustive -- there is only one way to leave out each line, or each donor. Nothing to reshuffle, so `repeat=0` for every fold and the reported SD is `NaN`, not zero. Their numbers are single estimates and are not directly comparable to the repeat-averaged schemes.
- **LOCO cannot support per-fold metrics at all** (a one-line test set has no ROC-AUC), which is the clearest illustration of why metrics are pooled within a repeat rather than averaged across folds.

Flat vs nested: which is used for what

Both are computed for every scheme, but only one is ever quoted as performance. Per task, `results/results_{task}.csv` holds:

tuning	rows	what a row is	used for
flat	440	one (scheme, model, k, param) combination	the robustness grid, and picking the config for the importance tables
nested	8	one (scheme, model)	every reported performance number

(440 = 4 schemes x 2 models x 11 k x 5 regularisation values; 8 = 4 schemes x 2 models.)

Every performance figure in this README is nested. R2 0.653, AUC 0.947, the all-schemes table -- all nested. Flat scores are optimistically biased, because the same predictions are used both to choose the winning configuration and to report its score. They are never quoted as performance.

But the feature-importance tables take their configuration from the FLAT grid (`feature_importance.py`, `select_winning_config_one_se`, which filters `tuning == "flat"`). That looks inconsistent and is deliberate:

Nested CV does not produce *a* configuration. It produces 50 of them -- one per outer fold -- and they disagree substantially (the modal choice wins only ~9-10 of 50; see "Hyperparameter selection is unstable"). There is nothing to read a coefficient off.

So to fit one set of coefficients and SHAP values you need one fixed model, and the one-SE rule scans the flat grid to choose it defensibly. Using flat here is safe because **the flat score is never reported** -- it is used only to *rank* configurations, not to claim performance.

The consequence to keep in mind: **the importance tables and the headline numbers describe slightly different models.** The "Configurations selected" table at the bottom lists both side by side (modal nested vs. one-SE flat) precisely so the gap is visible rather than implicit.

How nested CV actually runs (one repeat)

```

138 cell lines
|
├─ Outer fold 0: hold out 28 lines
|   └─ Use the remaining 110 for inner CV
|       └─ Inner fold 0: ~74 train / ~36 validate -> score every candidate
|           └─ Inner fold 1: ~74 train / ~36 validate -> score every candidate
|               └─ Inner fold 2: ~74 train / ~36 validate -> score every candidate
|                   └─ Average the 3 inner scores per candidate; pick the best (raw argmax --
|                       the one-SE rule is NOT applied here, see "What the one-SE rule is")
|                       └─ Discard the three inner models
|                       └─ Refit ONE model on all 110 outer-training lines
|                           └─ Predict the 28 outer-test lines ONCE
|
├─ Outer fold 1: 28 held-out lines predicted once
├─ Outer fold 2: 27
├─ Outer fold 3: 27
└─ Outer fold 4: 28
    ───
Final out-of-fold predictions: 28+28+27+27+28 = 138, each line exactly once

```

The 28 outer-test lines are invisible for the entire inner loop -- candidate scoring, selection and refit all happen inside the 110. Outer fold 0 contributes 28 predictions, not 3 or 55.

Per model, per outer fold: 55 candidates x 3 inner folds = **165 inner fits**, then 1 refit. Across 50 outer folds and 2 model families: **16,500 inner fits + 100 refits** per task.

Totals for one reported number:

outer fitted models	5 folds x 10 repeats = 50
final prediction values	138 lines x 10 repeats = 1,380
metrics computed	10 (one pooled AUC/R2 per repeat)
reported	mean +/- SD across those 10

How flat CV runs -- and one thing it does NOT do

Flat CV does not select a config per fold and then compare folds. There is no "best config for fold 0". Each candidate is scored the same pooled way the headline is:

```
for each of the 55 candidates:
  for each repeat (10):
    predict all 5 folds' held-out lines -> pool -> 138 predictions
    -> ONE score for that (candidate, repeat)
  -> 10 scores -> mean +/- SD across repeats
compare the 55 candidates by that mean; apply the one-SE rule once
```

So a flat row's `mean +/- SD` is across the **10 repeats**, exactly like a nested row -- not across the 5 folds and not across the 28 lines in a fold. The config is chosen **once**, from 55 means, not 5 times and then reduced.

The only structural difference from nested is *where the choice happens*: nested chooses inside each outer fold (so the choice never sees the test lines); flat chooses once, afterwards, over all the data. That is precisely why the flat score is optimistically biased and is never reported.

Best flat-CV configurations (donor_grouped)

Selection metric is MAE for regression and ROC-AUC for classification; R2 and AUC shown for readability.

model	raw argmax	one-SE choice	configs within 1 SE
ridge	k=5, alpha=0.01 (R2 0.6801)	k=5, alpha=0.1 (R2 0.6804)	2/55
lasso	k=5, alpha=0.001 (R2 0.6838)	k=5, alpha=0.001 (R2 0.6838)	2/55
logistic_l2	k=6, C=0.01 (AUC 0.9638)	k=4, C=0.01 (AUC 0.9629)	2/55
logistic_l1	k=4, C=0.1 (AUC 0.9640)	k=2, C=0.1 (AUC 0.9632)	9/55

The one-SE rule only moves the answer where the band is genuinely wide: `logistic_l1` has 9 statistically indistinguishable configs so it drops from k=4 to k=2, and `logistic_l2` from k=6 to k=4. For ridge and lasso only 2 configs qualify, so it barely bites.

Reconciling the two: they produce different things

The tension is real -- nested CV already does model selection, so why also run flat? Because they answer different questions and yield different kinds of output.

	question	output	score trustworthy?
nested	how well does <i>the procedure</i> generalise?	a number	yes -- this is the reported performance
flat	which single configuration ranks best?	a choice	no -- never quoted

Nested CV evaluates the whole pipeline *including its own tuning step*, which is what makes it honest. That same property is why it **cannot hand you a model**: it fits 50, each choosing its own config, and they disagree (the modal choice wins only ~9-10 of 50). There is no single "the nested model" to read a coefficient off.

So the division is:

Performance claims come from nested. Configuration choices come from flat. No number is ever taken from flat.

Being straight about the residual nuance: the flat ranking is computed over all 138 lines, so the chosen config has seen every outcome. For the coefficients and SHAP that is fine -- they are descriptive summaries of a full-data fit, not performance claims. For the LOCO and permutation deltas it is slightly less clean: those are measured out-of-fold but at a config chosen

using all the data, a mild optimism. It largely cancels because the deltas are within-fold with/without comparisons at the same config, but "largely cancels" is not "does not exist". The alternative -- using the modal nested config -- is also defensible and close (regression: one-SE k=5 vs modal k=4); see "Configurations selected". The one-SE flat config was chosen because a mode that wins 9-10 times out of 50 is a weak summary.

What donor_grouped actually does

Two things it is **not**: it does not put the same donors in every fold, and it does not average lines within a donor.

One row is always one cell line. 138 rows, 138 predictions. The only averaging is pool-then-line on the features (mean per (cell_line, pool), then unweighted mean across that line's pools). Donors are never collapsed; weighting is equal per line, not per donor.

What it does: **every line from a given donor lands in the same fold**, so a donor is entirely in train or entirely in test, never split across the two. Different folds hold out different donors.

folds.py passes groups=lines["donor"] to StratifiedGroupKFold. From the committed assignments, repeat 0 (138 lines, 20 donors, 5 folds):

fold	test lines	from donors
0	28	2
1	28	5
2	27	3
3	27	5
4	28	5

Donors appearing in both train and test of the same fold: 0.

Note fold 0 holds out 28 lines from only 2 donors -- that is the donor with 18 lines. Folds are balanced by line count, not donor count.

Why: 136 of 138 lines share a donor with another line. Without grouping a model can memorise a donor's genotype from one line and be scored on its sibling. Grouping blocks that; it costs ~0.012 R2, which is the number quantifying donor leakage.

Not to be confused with **LODO**, which is one fold per donor (20 folds, each holding out that donor's entire set of lines).

Leakage safety

- PCA/HVG refit **per fold**: exclude held-out lines' (or held-out donors' lines') cells from the fitting pool; project their cells into that fold's PCA space. Generalizes 005_d11_pca_features.py's fit_mask beyond just pool11.
- **pool11 rule frozen** from EDA (depth ~1,900 vs ~10-18K elsewhere) — always excluded from fitting when present in training, never tuned on results. Confirmed harmless: 4.2% of D11 cells (9,603/227,983); ~218K remain.
- **Pool correction**: residualize each feature on pool identity using that fold's *training* pool means, computed at the (cell_line, pool) level before line-averaging (slots into the existing 004 / 005 pool→line pipeline). Both corrected/uncorrected variants run.
- **No donor feature-correction** — donor is likely real signal, not nuisance; donor-grouping is the right tool, correcting it out risks deleting the signal we want.
- Known unfixable limitation: D11/D52 cell-type labels came from the paper's own global clustering, not refit per fold — diffuse, minor, unavoidable.

Metrics

Two different thresholds, easily confused. 0.2 is the *outcome* threshold that turns diff_efficiency into the success/failure label -- it defines the question. 0.5 is the *decision* threshold that turns a predicted probability into a hard call (harness.DECISION_THRESHOLD). Sensitivity, specificity, balanced accuracy and F1 are all computed at 0.5, not 0.2. (An earlier version of this line said "@ 0.2", which conflated exactly the two things the next sentence warned about.)

Classification

metric	what it is	what it is sensitive to	trivial baseline
ROC-AUC	P(a random success scores above a random failure)	ranking only; threshold-free and unaffected by where you set the cutoff	0.500
PR-AUC	area under precision-recall; ignores true negatives entirely	the positive class; false alarms cannot hide behind a big negative class	prevalence, 0.696
balanced accuracy	(sensitivity + specificity) / 2 at the 0.5 cutoff	both classes equally, regardless of imbalance	0.500
sensitivity	of true successes, the fraction called success	misses on the majority class	1.000
specificity	of true failures, the fraction called failure	misses on the minority class -- the one that matters here	0.000
F1	harmonic mean of precision and recall, success as positive	the positive class only; ignores true negatives	0.821
Brier	mean squared error of the predicted probability (lower better)	<i>calibration</i> -- whether a "0.8" means 80%	0.212

Why this set rather than one number. Each covers a failure the others miss:

- **Accuracy is disqualified outright.** 70% of lines succeed, so guessing "success" every time scores 0.70. Balanced accuracy replaces it.
- **ROC-AUC checks ranking, Brier checks the numbers.** A model can rank perfectly (AUC 1.0) while every probability is wrong -- and these probabilities matter, because the useful operating point is chosen by moving the threshold. Only Brier catches that.
- **PR-AUC covers ROC-AUC's blind spot at imbalance.** With 96 negatives available, false positives get diluted in the false-positive rate but not in precision.
- **Sensitivity and specificity are reported separately,** not just as their average, because they fail asymmetrically. The interesting class here is the 42 failures, and specificity is the metric that tracks them -- balanced accuracy alone would hide a specificity collapse behind high sensitivity.
- **F1 is reported for convention, not evidence.** Its trivial baseline is **0.821** on this data, so a "good" F1 of 0.945 is far less impressive than it looks. It is in the table mainly so nobody has to recompute it.

Every baseline is in the results table so each metric can be read against what guessing achieves, rather than against 0 or 1.

Regression

metric	what it is	why it is here	trivial baseline
R2	fraction of outcome variance explained	the conventional summary; comparable across datasets	0.000
MAE	mean absolute error, in efficiency units	directly interpretable -- "typically wrong by 0.138 efficiency"	0.266
RMSE	root mean squared error	penalises large misses; RMSE >> MAE flags a few big errors rather than uniform mediocrity	0.298
out-of-range	fraction of predictions outside [0, 1]	a sanity check: the outcome is a proportion, and a linear model is not constrained to produce one	0.000

MAE and RMSE are both reported because the gap between them is informative. Here 0.138 vs 0.175 is a moderate ratio -- errors are fairly uniform, not driven by a handful of catastrophic misses.

R2 is the one to distrust here. It is inflated by the bimodal outcome: most of the 0.653 comes from separating the failure clump from the success clump, not from precision within either. Within-success R2 is ~0.17-0.22 and within-failure R2 is about -14. MAE 0.138 against a failure clump whose entire range is 0.173 makes the same point in interpretable units. Inner-loop tuning therefore selects on **MAE**, not R2.

Reproducibility

- Fixed seeds; persist actual fold assignments (line IDs per scheme × repeat × fold) to a file, not just the seed.

- `models.SEED` pins `random_state` on both logistic factories. `saga` (used by `logistic_l1`) is stochastic; unseeded it drifted $\sim 5.7e-4$ between runs, which showed up as spurious 4th-decimal diffs in regenerated importance tables.
- Fold features are written incrementally per fold and a run can be resumed; see the D30 section for the provenance gap that makes resume unsafe across a settings change.

Scope/cost warning

Full parallel grid (2 groupings $\times$ 2 tuning $\times$ 2 corrections $\times$ 2 model families, repeated K-fold, + LOCO/LODO) means potentially hundreds of fold-specific PCA refits (~ 90 s each in `005`). Time one fold first; cache fold-specific fits before committing to the full run.

Results distillation

"**Headline model**" = the single configuration reported as *the result*, as opposed to the robustness grid of everything that was run. Without one there are 16 numbers per task (4 schemes $\times$ 2 tuning modes $\times$ 2 model families) and no answer -- and whichever gets quoted, a reader cannot tell whether it was chosen before or after seeing the scores.

It is **pre-registered, not selected on performance**. Committed as constants in `run_experiment.py` and enforced by `select_headline()`:

```
HEADLINE      = {"scheme": "donor_grouped", "tuning": "nested", "pool_correction": False}
HEADLINE_MODEL = {"regression": "ridge", "classification": "logistic_l2"}
```

Each choice was made on design grounds, not on scores: `donor_grouped` because it is the strictest realistic test; `nested` because flat CV tunes and reports on the same rows; no pool-correction per the investigation below; `ridge` / `logistic_l2` as the L2 default, with L1 as the variant.

Note the L1 variants score *higher* (lasso R2 0.669 vs ridge 0.653; `logistic_l1` AUC 0.951 vs 0.947). A paired per-repeat comparison at matched one-SE configs shows lasso's edge is real but tiny -- $+0.0033$ R2, winning 9/10 repeats, t-test $p=0.0006$. Most of the larger headline gap is nested CV's per-fold reselection noise, not a model difference. They are reported as labelled secondary results; quoting them as "the" headline is exactly the cherry-picking the pre-registration exists to prevent, and that failure actually occurred once -- see "Corrections from the audit" #3.

- Full grid still gets computed, but pre-register ONE headline config *before* looking at results, to avoid cherry-picking: **donor-grouped + nested CV + repeated stratified K-fold + Ridge/L2-logistic** (pool-correction dropped entirely -- see "Pool-correction dropped" above). Report this first, with mean $\pm$ SD.
- Everything else (plain grouping, flat CV, Lasso/L1, LOCO/LODO) becomes a secondary "robustness grid" — one table/heatmap (rows = config, cols = key metric) to scan for consistency, not N separate headline results.

Architecture (Open-Closed + reusable for D30→D52)

- **Model registry, not conditionals**: a list of `ModelSpec(name, estimator_factory, param_grid)` entries (Ridge/Lasso/L2-logistic/L1-logistic to start). The CV harness only ever calls the sklearn estimator interface (`.fit` / `.predict` / `.predict_proba`) generically — adding a model later (e.g. random forest) = one new registry entry, zero changes to harness code.
- **Feature extraction (timepoint-parameterized) vs. CV harness (timepoint-agnostic)** — *aspirational*; see "D30 readiness" above for what actually still hardcodes D11: feature extraction takes a timepoint (D11 now, D30 later — same 138 qualifying lines, since the qualifying list already requires ≥ 10 cells at D11 and D30 and D52) and a fold's train/held-out line split, returns feature matrices. The CV/tuning/metrics harness takes `X_train`, `y_train`, `X_test`, `y_test` + the model registry and is completely agnostic to which timepoint produced the features. D30→D52 later = swap the feature-extraction call, harness untouched.

Files

- `features.py` — feature-extraction, parameterized by timepoint (D11 now; reusable for D30).
- `folds.py` — fold construction (plain/donor-grouped repeated stratified K-fold, LOCO, LODO) — timepoint-agnostic.
- `models.py` — model registry (`ModelSpec` list).
- `harness.py` — CV/tuning/metrics harness using the registry + fold module.
- `run_regression.py` / `run_classification.py` — orchestration entry points.
- Summary table + comparison plots (headline config; plain vs. donor gap; flat vs. nested; corrected vs. uncorrected).

Verify

- Per-fold fitting cell counts stay large; fold assignments reproducible; classification metrics only where both classes present; repeated-CV reported per-repeat not pooled; regression predictions checked against [0,1].

Latest results

Authoritative current numbers. Regenerated 2026-09-09 in a single clean run on one consistent code state (after the seed pin and the fixes in "Second correctness review"), so no figure here is a mix of pre- and post-fix runs. Everything below is the pre-registered configuration: **donor_grouped grouping, nested CV, no pool-correction**.

Headline (pre-registered model per task)

Nested CV, no pool-correction, mean +/- SD across 10 repeats. Columns are the two repeated schemes: `plain` ignores donors, `donor_grouped` never lets a donor appear in both train and test.

Every metric the harness computes, both repeated schemes, headline and secondary models together. See "Metrics" for what each one means.

Classification

model	scheme	ROC-AUC	PR-AUC	balanced acc	sensitivity	specificity	F1	Brier
logistic_l2 (headline)	plain	0.940 +/- 0.012	0.964 +/- 0.011	0.914 +/- 0.013 ★	0.931 +/- 0.016	0.898 +/- 0.023 ★	0.943 +/- 0.009	0.102 +/- 0.006
logistic_l2 (headline)	donor-grouped	0.947 +/- 0.007	0.968 +/- 0.004	0.913 +/- 0.013	0.940 +/- 0.007	0.886 +/- 0.022	0.945 +/- 0.007 ★	0.097 +/- 0.008
logistic_l1 (secondary)	plain	0.948 +/- 0.011	0.968 +/- 0.012	0.904 +/- 0.019	0.933 +/- 0.005	0.874 +/- 0.037	0.939 +/- 0.009	0.077 +/- 0.007
logistic_l1 (secondary)	donor-grouped	0.951 +/- 0.008 ★	0.970 +/- 0.009 ★	0.912 +/- 0.013	0.942 +/- 0.011 ★	0.883 +/- 0.021	0.945 +/- 0.008 ★	0.075 +/- 0.004 ★
trivial baseline	--	0.500	0.696	0.500	1.000	0.000	0.821	0.212

Regression

model	scheme	R2	MAE	RMSE	out-of-range
ridge (headline)	plain	0.676 +/- 0.013	0.134 +/- 0.003 ★	0.170 +/- 0.003	0.050 +/- 0.007
ridge (headline)	donor-grouped	0.653 +/- 0.021	0.138 +/- 0.004	0.175 +/- 0.005	0.043 +/- 0.005 ★
lasso (secondary)	plain	0.680 +/- 0.016 ★	0.134 +/- 0.003 ★	0.169 +/- 0.004 ★	0.049 +/- 0.008
lasso (secondary)	donor-grouped	0.668 +/- 0.015	0.135 +/- 0.003	0.172 +/- 0.004	0.046 +/- 0.004
predict the mean	--	0.000	0.266	0.298	0.000

★ = best in that column among the four fitted rows (baselines excluded). **Bold** marks the pre-registered headline row, which is a separate thing from being best. Higher is better everywhere except **Brier**, **MAE**, **RMSE** and **out-of-range**, where lower is better -- so the star follows the minimum in those columns.

Two things the full table shows that the headline number alone does not:

- **L1 is better calibrated than L2** (Brier 0.075 vs 0.097, a 23% lower squared probability error) even though their AUCs are within one SD. `class_weight="balanced"` pushes L2's probabilities away from the true base rate more than L1's; ranking is unaffected, calibration is not.
- **4-5% of regression predictions fall outside [0, 1]** -- impossible values for a proportion. A linear model has no constraint keeping it in range, and this is another reason to use the classifier for decisions.

Donor grouping costs regression a little and classification nothing. Ridge loses 0.023 R2 (0.676 -> 0.653) and lasso 0.011 (0.680 -> 0.669). Both classification models are flat or slightly *better* under grouping (L2 0.940 -> 0.947, L1 0.948 -> 0.951), which is the same conclusion the LOCO/LODO contrast gives: donor leakage is real but small for regression and absent for classification.

The PCA fitting population barely matters. Every number above uses the baseline basis. Refitting the whole pipeline on the qualifying-cells-only basis moves the donor-grouped results by:

model	delta
ridge	+0.0141 R2
lasso	+0.0040 R2
logistic_l2	+0.0013 AUC
logistic_l1	-0.0013 AUC

All within about one SD, and only ridge moves by more than a rounding error. The variant does roughly halve regression's across-repeat SD (0.021 -> 0.012), so it is a little more stable, but no conclusion changes either way. Full comparison in "Fitting population: the qualifying-only PCA variant".

Configurations selected

Nested CV re-selects per outer fold, so `modal` is the most common choice across 50 folds, not "the" configuration -- the spread is wide (see "Regenerated results" note below). The one-SE column is the single config used for the feature-importance tables.

run	model	modal nested config	one-SE config (importance tables)
baseline	ridge	k=4, alpha=0.01	k=5, alpha=0.1
baseline	lasso	k=5, alpha=0.001	k=5, alpha=0.001
baseline	logistic_l2	k=2, C=100.0	k=4, C=0.01
baseline	logistic_l1	k=2, C=1.0	k=2, C=0.1
qualifying-only	ridge	k=6, alpha=1.0	k=6, alpha=0.1
qualifying-only	lasso	k=6, alpha=0.001	k=6, alpha=0.001
qualifying-only	logistic_l2	k=8, C=0.01	k=8, C=0.01
qualifying-only	logistic_l1	k=0, C=0.1	k=3, C=0.1

Note `logistic_l1` selects **k=0** most often on the qualifying-only basis -- i.e. the modal nested model uses the cell-type proportions and no PCs at all, and still reaches AUC 0.950. Consistent with the standing finding that the proportions (`phat_NB` especially) carry most of the classification signal.

Hyperparameter selection is unstable -- read single-config tables with that in mind

Recovering the per-fold selections (they were previously computed and dropped) shows no configuration dominates. Across 50 outer folds the modal choice wins only ~9-10 times, spread over 13-19 distinct configs, with `C` ranging the full four orders of magnitude. Full per-fold record in `results/nested_selections_{task}{suffix}.csv` (516 rows each). This is why the headline is pre-registered and why the one-SE rule is used -- and why any single-config coefficient table is one draw from a wide distribution.

Files backing this section

	baseline	qualifying-only
grids	results/results_{task}.csv	results/results_{task}_qualonly.csv
per-fold selections	results/nested_selections_{task}.csv	results/nested_selections_{task}_qualonly.csv
importance	results/feature_importance_table_{task}_{model}.csv	results/..._{model}_qualonly.csv
fold features	fold_data/fold_features_D11_full.csv	fold_data/fold_features_D11_qualonly.csv
global PCA basis	metadata_eda/pca/d11_pca_coords_per_line.csv	..._qualonly.csv

EDA outputs and provenance

source: `metadata_eda/README.md`

What each EDA script produced, why, and the caveats found along the way.

metadata_eda

Project-level findings summary with literature context: [FINDINGS.md](#)

Outputs from the EDA scripts (`001_eda.py`, `002_metadata_eda.py`, `003_qualifying_cell_lines.py`, in the repo root) run against the Jerber et al. dopaminergic neuron differentiation dataset (<https://pmc.ncbi.nlm.nih.gov/articles/PMC7610897/>): `day11.h5`, `day30.h5`, `day52.h5`.

Layout

```
metadata_eda/
  cohort/      who is in the study: qualifying (cell_line, pool) combos,
               the D52 label, per-line/pool/timepoint cell counts
  pca/         D11 PCA outputs -- per-line and per-cell coordinates, gene
               loadings, variance explained, HVG lists.
               `*_qualonly` = the qualifying-cells-only fitting variant
  proportions/ D11 cell-type proportions + their sampling SE
  qc/          QC summaries and metadata crosstabs
  technical/   technical-covariate (pool/batch) association analyses
  plots/       all figures
```

`cohort/qualifying_cell_line_pool_min10_per_timepoint.csv`, `cohort/d52_diff_efficiency_label.csv`, `proportions/d11_celltype_proportions_with_se.csv` and `pca/d11_pca_coords_per_line.csv` are the interface consumed by `modeling/` -- changing their paths means updating the modeling package.

Source metadata fields (in the `.h5` files themselves)

`obs` columns, beyond `celltype`:

- `donor_id` — full iPSC line identifier (e.g. `HPSI0114i-eipl_1`); parsed into `donor` (the `HPSI####i` prefix) and the specific clonal line
- `pool_id` — which multiplexed differentiation pool (7-24 lines each) the cell came from
- `sample_id` / `sample_index` — the specific 10x sequencing sample within a pool (finer-grained than pool)
- `time_point` — D11 / D30 / D52
- `treatment` — `NONE` or `ROT` (rotenone oxidative-stress exposure, D52 only)
- `cluster_id` — a plain numeric per-timepoint clustering assignment (e.g. Leiden/Louvain cluster number), distinct from the annotated `celltype` label

Beyond `obs`:

- `obsm/X_umap` — precomputed 2D UMAP coordinates per cell, per timepoint
- `var` — gene symbols (`var/index`, 32,738 genes, identical set/order across all three files) and, for day11/day30 only, redundant per-sample Ensembl gene IDs (`var/gene_ids-0..N`)
- `X` / `raw/X` — the expression features themselves: `X` is log-normalized expression (float32), `raw/X` is raw UMI counts (float32-encoded integers), both genes x cells sparse matrices

No other per-cell covariates (e.g. sex, age, batch date, sequencing depth as a stored field) are present in `obs` — total counts / genes detected had to be computed from `raw/X` directly (see `qc/qc_summary_by_timepoint_celltype.csv` below).

From `001_eda.py`

- `cohort/cell_line_donor_pool_timepoint_n_cells.csv` — `n_cells` grouped by `donor`, `cell_line`, `timepoint`, `pool`.
- `cohort/cell_line_donor_pool_timepoint_celltype_n_cells.csv` — same, with `celltype` added as an extra grouping column.

From `002_metadata_eda.py`

- `qc/metadata_crosstab_treatment.csv` — `n_cells` by `timepoint` x `treatment`. Surfaces that `treatment` is `NONE` for D11/D30 but `NONE` / `ROT` for D52 (the paper's rotenone oxidative-stress condition).
- `qc/metadata_crosstab_celltype.csv` — `n_cells` by `timepoint` x `celltype`.
- `qc/metadata_crosstab_celltype_by_cluster.csv` — `n_cells` by `timepoint` x `cluster_id` x `celltype`, to see how the numeric Leiden/Louvain `cluster_id` maps onto annotated cell types.
- `qc/qc_summary_by_timepoint_celltype.csv` — `describe()` (count, mean, std, min/25/50/75/max) of `total_counts` and `n_genes_detected` per cell, grouped by `timepoint` x `celltype`. These two per-cell metrics are computed from `raw/X` (raw UMI counts), streamed in 20k-cell chunks so the full multi-GB sparse arrays are never loaded at once.
- `plots/plot_celltype_pool_bars.png` — bar charts of `n_cells` by `celltype` and by `pool`, grouped by `timepoint`.
- `plots/plot_umap_by_celltype.png` — per-timepoint UMAP scatter (using each file's precomputed `obsm/X_umap`), colored by `celltype`. Downsampled to at most 50,000 cells per timepoint for plot legibility/speed.
- `plots/plot_qc_distributions.png` — histograms of `total_counts` (log10) and `n_genes_detected` per cell, overlaid by `timepoint`.

From `003_qualifying_cell_lines.py`

- `cohort/qualifying_cell_line_pool_min10_per_timepoint.csv` — every (`cell_line`, `pool`) combination with ≥ 10 cells in that *same* pool at all three timepoints (D11, D30, D52). 159 combos, spanning 138 distinct cell lines (some lines qualify through more than one pool).

Filtering matches the method in the paper's own analysis notebook

(`plotting_notebooks/Figure_2/fig2b_and_heatmap_extended.ipynb` in https://github.com/single-cell-genetics/singlecell_neuroseq_paper), which reports "138 lines": count cells per (`cell_line`, `pool`) at each timepoint, keep combos with ≥ 10 cells, then keep only combos that survive at all three timepoints. An earlier version of this script summed a cell line's cells across *different* pools before filtering, which is looser and over-counted (140 lines) — a cell line split as, e.g., 6 cells in one pool and 8 in another at a timepoint would incorrectly pass.

From `004_d11_celltype_proportion_se.py`

Sanity check for a candidate feature (predicting D52 differentiation efficiency from D11 state). **Each data point is a `cell_line`**, restricted to the 138 lines in `cohort/qualifying_cell_line_pool_min10_per_timepoint.csv` (the paper-matching set). A line qualifying through more than one pool has its D11 cell type proportion **averaged across those pools** (the authors' approach — 21 of the 138 lines average across 2+ pools):

```
p_i = mean(phat_k for each qualifying pool k of line i)
```

Averaging proportions (rather than pooling raw counts) means `SE_i` is the standard error of a mean of independent estimates, not a single bigger-sample binomial SE:

```
SE_i = sqrt(sum_k SE_k^2) / n_pools_i, SE_k = sqrt(phat_k*(1-phat_k)/n_k)
```

(a line with only one qualifying pool has `n_pools_i=1`, reducing to the plain binomial SE of that pool). The question: is the variance of a D11 cell type's proportion observed *across cell lines* bigger than the sampling variance expected *within* each line, or is the apparent spread just counting noise?

- `proportions/d11_celltype_proportions_with_se.csv` — per-line `n_pools`, `phat` / `se` (the `p_i / SE_i` above) for each D11 celltype (FPB, P_FPB, NB).
- `proportions/d11_celltype_proportion_variance_vs_se.csv` — per-celltype summary: `observed_std` (std of `p_i` across lines, i.e. `sqrt(Var(p_1, p_2, ...))`) vs. `rms_se` (`sqrt(mean(SE_i^2))`) — the sampling-noise SD on the same scale; not the plain mean of `SE_i`, plus `variance_ratio = Var(p_i) / mean(SE_i^2)`. All three D11 celltypes have `observed_std` 4-8x `rms_se` (variance_ratio 14-58x) — real line-to-line signal clearly dominates counting noise, so D11 cell type proportions look like a reasonable feature to use.
- `plots/plot_d11_celltype_proportion_vs_se.png` — per celltype, `phat` per cell line (sorted, with `se` error bars) vs. a dashed line at the mean; visually, the spread across lines is far wider than any individual error bar.

From `005_d11_pca_features.py`

Third candidate feature: per-cell_line PCA coordinates from D11 single-cell expression. Pipeline: take all D11 cells -> use the file's existing log-normalized `X` (already normalized; no renormalization done) -> select 2000 HVGs (genes binned into 20 bins by mean expression, top genes by within-bin z-scored dispersion — a simplified variant of Scanpy's `flavor='seurat'`, operating directly on log values) -> scale each HVG to zero mean/unit variance and clip at ± 10 -> PCA to 10 components -> every D11 cell gets `PC1..PC10` -> per cell line, mean PCs per (`cell_line`, `pool`) first, then mean of those pool-means (matching `004`'s averaging approach), restricted to the 138 lines in `cohort/qualifying_cell_line_pool_min10_per_timepoint.csv`.

Important caveat, found and corrected during this analysis: `pool11` is a severe sequencing-depth batch outlier ($\sim 1,900$ mean UMI/cell vs. $\sim 10,000$ - $18,000$ in every other pool). Fitting PCA on all D11 cells made PC1 almost entirely a "is this pool11" indicator (17% of variance, every pool11 line pinned to one extreme, every other line near the mean) rather than a differentiation-relevant axis — and since 15 of `pool11`'s 16 qualifying lines have no other qualifying pool, simply dropping `pool11` would have shrunk the feature table from 138 to 123 lines and mismatched the `004` feature table. Fix: HVG selection and PCA fitting (mean/variance for scaling, and the PCA components themselves) use only the 243,623 non-`pool11` D11 cells; `pool11` cells are then *projected* into that learned space (`PCA.transform`, not `.fit_transform`) so all 138 lines still get PC coordinates. After the fix, PC1 explains only 5.4% of variance (vs. 17% before) with a smooth scree decay, and the PC1-PC2 scatter of line-level means shows a continuous cloud, not a pool11 cluster. A residual, much smaller batch signature remains on PC8 (0.85% of variance) where `pool11` lines still show higher spread — worth keeping in mind if using the later PCs for modeling.

- `pca/d11_hvg_genes.csv` — the 2000 selected HVGs (gene symbol, index, mean, variance — from non-`pool11` cells).
- `pca/d11_pca_variance_explained.csv` — explained variance ratio per PC.
- `pca/d11_pca_coords_per_line.csv` — the 138-line x (`PC1..PC10`) feature table.
- `plots/plot_d11_pca_scree.png` — explained variance ratio per PC.
- `plots/plot_d11_pca_scatter.png` — PC1 vs PC2 of the 138 line-level means.

From `006_d11_pca_variance_vs_se.py`

Sanity check for the PCA feature, same idea as `004`'s check for cell type proportions but generalized from a binomial proportion to a continuous per-cell value: a line's `PC_k` feature is the mean of many single-cell `PC_k` values, so its standard error is the standard error of that mean, `SE_i = std(cell-level PC_k in line i) / sqrt(n_i)` (exactly analogous to `SE_i = sqrt(p_i*(1-p_i)/n_i)`, which is also an SE-of-a-mean, just of 0/1-valued cells). Same two-stage pool-then-line averaging and SE propagation as `004` / `005`.

Also adds an `icc` column: `icc = 1 - 1/variance_ratio`, a bounded (0-1) reframing of the same ratio (`observed_var` already includes averaged sampling noise, so `variance_ratio`'s "no signal" floor is 1, not 0; `icc` maps that to 0, and -> 1

for strong signal) — the same quantity as `variance_ratio`, just easier to read.

Result, and a caveat found while checking it: every PC looks extremely reliable at face value (`variance_ratio` 24-2038x, `icc` 0.96-0.9995) — but PC8 and PC9's numbers are almost entirely a residual `pool11` artifact (the sequencing-depth batch outlier from `005`): excluding the 16 `pool11`-derived lines drops their `variance_ratio` by 30-60x (2038 -> 32, 1047 -> 49; see `variance_ratio_excl_pool11` / `icc_excl_pool11` columns and the sharp, isolated jump for `pool11` lines in `plots/plot_d11_pca_variance_vs_se.png`'s PC8/PC9 panels). This is expected: a technical batch that shifts every one of its cells the same way is *reliably different* by this test, indistinguishable from real biology using variance-vs-noise alone. PC1/PC2 are more robust (24 -> 12, 663 -> 389 when excluding `pool11`) but still somewhat inflated. All PCs remain `> 1` even excluding `pool11` (12-390x), so all 10 look like real, reliable signal — just noting that the *size* of that reliability shouldn't be read at face value for PC8/PC9 specifically, and that this check only establishes reliability, not that a PC predicts D52 efficiency (a separate question for later cross-validated regression).

- `pca/d11_pca_line_level_with_se.csv` — per-line `n_pools` and, for each PC, `{PC}_mean` / `{PC}_se` (the `p_i` / `SE_i` above).
- `pca/d11_pca_variance_vs_se.csv` — per-PC summary: `observed_std`, `rms_se`, `variance_ratio`, `icc`, plus `variance_ratio_excl_pool11` / `icc_excl_pool11`.
- `plots/plot_d11_pca_variance_vs_se.png` — one panel per PC, each line's value sorted with an SE error bar.

From `007_d11_cell_counts_per_line.py`

How many D11 cells does each cell line have — with a per-pool breakdown for lines profiled in more than one pool? Uses `cohort/cell_line_donor_pool_timepoint_n_cells.csv` (from `001`), so covers every D11 cell line (177), not just the 138 qualifying ones. Also flags which lines are among those 138, for cross-reference — a line can have plenty of D11 cells and still not qualify, since qualifying also requires

= 10 cells in the *same pool* at D30 and D52.

- `cohort/d11_cell_counts_per_line.csv` — per line: `donor`, `n_pools_D11`, `pool_breakdown_D11` (e.g. `pool4:2125`, `pool5:8321`), `total_n_cells_D11`, `is_qualifying_138`. Sorted by `total_n_cells_D11` descending. 25 of the 177 lines span more than one pool at D11; totals range from 1 to 14,640 cells.
- `plots/plot_d11_cell_counts_per_line.png` — sorted bar chart, colored by qualifying status.

From `008_technical_covariate_associations.py`

Formalizes the ad hoc `pool11` finding (`006`) into a systematic correlation/association screen. Recomputes the **uncorrected** PCA (fit on *all* D11 cells, i.e. the pre-`005`-fix version that `pool11` dominated) and checks it — plus the `004` cell type proportions, which had never been checked this way — against technical covariates at the 159-combo granularity: `mean_total_counts` / `mean_n_genes_detected` (Pearson r , from `002`'s QC metrics), `n_cells` (Pearson r), and `pool` identity (η -squared / one-way-ANOVA variance-explained — the direct generalization of "PC1 grouped by pool").

Headline result: as expected, uncorrected PC1 is almost entirely explained by `pool` ($\eta_{sq_pool}=0.999$) and strongly anti-correlated with sequencing depth ($r=-0.87$ with `mean_total_counts`, $r=-0.91$ with `mean_n_genes_detected`) — confirms the `005` / `006` `pool11` story quantitatively. PC5 and PC7 also show strong `pool` associations (η_{sq_pool} 0.87 and 0.83) not previously flagged; PC5's tracks sequencing depth the same way PC1's does ($r\approx-0.88$), while PC7's doesn't ($r\approx-0.03$) — a `pool` effect from something other than depth.

More important: the cell type proportions are substantially technical-covariate-confounded too, which hadn't been checked before. `phat_FPP` and `phat_P_FPP` correlate strongly with sequencing depth ($r=0.73$ / $r=-0.72$ with `mean_total_counts`; $r=0.79$ / $r=-0.75$ with `mean_n_genes_detected`) and `pool` (η_{sq_pool} 0.79 and 0.69) — likely because cell type calls are sensitive to capture depth (fewer marker genes detected per cell biases classification). `phat_NB` looks clean ($\eta_{sq_pool}=0.03$, no significant correlations). This means the `004` proportions feature, while *reliable* (validated via ICC in `004` / `006`), is not free of the same technical confound found in the PCA feature — worth accounting for (e.g. as a covariate, or checking whether it survives controlling for `pool`/depth) before treating FPP/P_FPP as clean predictors of D52 efficiency.

- `technical/technical_covariate_correlations_pca_uncorrected.csv` — 10 PCs x $\{r, p\}$ per numeric covariate + `eta_sq_pool`.
- `technical/technical_covariate_correlations_celltype_proportions.csv` — same, for the 3 D11 celltypes.
- `plots/plot_technical_covariate_correlations.png` — correlation heatmaps (PCs and proportions vs. numeric covariates) and eta-squared-by-pool bar charts.
- `technical/technical_covariate_analysis_plan.md` — the design notes/plan written before implementing this and `007`.

From `009_d52_outcome_label.py`

The D52 outcome label: **differentiation efficiency** = fraction of a cell line's D52 cells that are DA (dopaminergic) or Sert (serotonergic) neurons, the two mature "successfully differentiated" types — matching the paper's own definition (`fig2b_and_heatmap_extended.ipynb`: `diff_efficiency = DA_D52 + Sert_D52`). DA and Sert are combined into one binary indicator *before* computing proportions/SE (not two proportions summed afterward), since they're mutually exclusive outcomes of the same per-cell draw. Otherwise identical methodology to `004`: binomial proportion + SE per (`cell_line`, `pool`), restricted to the 138 qualifying lines, two-stage pool-then-line averaging with SE propagation for multi-pool lines. Reuses `004`'s `compute_pool_level_proportions_and_se` and `collapse_to_line_level` directly (via `importlib`, same pattern as `008`).

138 lines get a label; wide spread (0.012 to 0.922, mean 0.45), with a notable cluster of ~20 lines near 0 (essentially fail to produce DA/Sert at all) and a broad spread from there — plenty of variance for a downstream model to explain, and SE error bars are small relative to that spread by eye.

- `cohort/d52_diff_efficiency_label.csv` — `cell_line`, `n_pools`, `diff_efficiency`, `diff_efficiency_se`.
- `plots/plot_d52_diff_efficiency.png` — sorted per-line values with SE error bars, and a histogram of the distribution.

From `010_d52_label_technical_covariates.py`

Same technical-covariate check `008` ran on the D11 features, applied to the D52 `diff_efficiency` label — matters because a shared confound on both sides (a D11 feature *and* the D52 label) would let a "predictive" model learn the confound instead of real biology, regardless of whether either side looks confounded alone.

Good news: the label looks clean. `diff_efficiency` has weak, non-significant correlations with D52 sequencing depth ($r=0.04$, $p=0.63$) and genes detected ($r=0.10$, $p=0.22$), and a much smaller `eta_sq_pool=0.13` than the D11 features saw (0.69-1.0). Notably, pool11 (the severe D11 sequencing-depth outlier) is unremarkable here — its `diff_efficiency` box (median ~0.68, IQR 0.53-0.85) sits in the middle of the pack, not off on its own. Pool medians do vary somewhat (0.25-0.68 across pools) but the boxes overlap heavily, consistent with the modest eta-squared rather than a dominant batch effect.

- `technical/technical_covariate_correlations_d52_label.csv` — the association numbers above.
- `plots/plot_d52_label_technical_covariates.png` — scatter of `diff_efficiency` vs. mean D52 sequencing depth, and a boxplot by pool.

Regenerating

From the repo root:

```
uv run python 001_eda.py
uv run python 002_metadata_eda.py # streams raw/X for QC metrics; slower (~1-2 min)
uv run python 003_qualifying_cell_lines.py
uv run python 004_d11_celltype_proportion_se.py # depends on 003's output CSV
uv run python 005_d11_pca_features.py # depends on 003's output CSV; slower (~1-2 min)
uv run python 006_d11_pca_variance_vs_se.py # depends on 005's per-cell-PC CSV
uv run python 007_d11_cell_counts_per_line.py # depends on 001 and 003's output CSVs
uv run python 008_technical_covariate_associations.py # depends on 003; slower (~1-2 min, recomputes
uncorrected PCA)
uv run python 009_d52_outcome_label.py # depends on 003's output CSV
uv run python 010_d52_label_technical_covariates.py # depends on 003's output CSV
```

All ten scripts write into this directory (creating it if needed) and read the source `.h5` files from the paths hardcoded in each script's `DATA_FILES`. `008`, `009`, and `010` also import functions directly from `002` / `004` / `005` / `008` via `importlib` (numbered modules aren't importable with a plain `import`) rather than duplicating their logic.